



RETINEXPLAIN

Explainable Retinal Disease Detection via Concept Bottleneck Models and Genetic Programming-Evolved Fuzzy Pattern Trees

A Dissertation submitted in partial satisfaction of the requirements for the degree "Master of Science in Artificial Intelligence & Machine Learning"

by

Ashik Kannampilly Janardhanan

ID Number: 25032615

Supervised by Dr. Douglas Mota Dias and Dr. Allan de Lima

Department of Computer Science and Information Systems
Faculty of Science and Engineering
University of Limerick

2026

Statement

I agree for this dissertation to be made available in the University of Limerick library and to future MSc students in the Department of Computer Science and Information Systems.

Abstract

Retinal fundus photography is a non-invasive procedure for detecting eye diseases such as diabetic retinopathy, glaucoma, and age-related macular degeneration. Though Deep Learning-based approaches demonstrate remarkable results in terms of classification accuracy, the lack of interpretability undermines the algorithm's reliability and contradicts the need for justification in diagnostic imaging.

The present dissertation introduces RETINEXPLAIN, an end-to-end explainable solution for multi-class retinal disease classification task implemented on the ODIR-5K dataset. It utilizes a frozen EfficientNet-B0 encoder for feature extraction, which are subsequently transformed into meaningful clinical concepts with the use of a Concept Bottleneck Layer. Those features along with the set of patient demographics are then subjected to fuzzy logic analysis performed by Genetic Programming-evolved Fuzzy Pattern Tree, which generates human-readable logical rules for classification.

On the held out test set, RETINEXPLAIN achieved a macro F1 of 0.4334, which is effectively tied in practical terms with the best performing interpretable model, a tuned decision tree (0.4344), with a difference of 0.0010 macro F1 well within the run-to-run variance of RETINEXPLAIN's own 30 independent GP runs, outperforming it in raw accuracy by a seven-point margin (56.63% vs 49.65%). It underperformed the non-interpretable models, including a tuned random forest (0.5003) and an end-to-end black box EfficientNet-B0 (0.5829), highlighting the accuracy cost of using an interpretable model. This accuracy cost is the price to pay for the medical utility of having each prediction trace to a human readable symbolic rule composed of named clinical concepts, unavailable in the accuracy of the black box models.

The major contribution of the present study lays in conceptualizing a novel combination of concept-based learning, fuzzy logic, and genetic programming for creating an explainable diagnostic imaging solution while demonstrating competitive performance in comparison to the state-of-the-art methods.

Acknowledgements

I would like to thank my supervisors, Dr. Douglas Mota Dias and Dr. Allan de Lima, for their guidance, patience and detailed feedback throughout this project. Their willingness to go through my code, debug my errors and challenge my rationale greatly helped in producing this paper. Furthermore, I would like to thank the Department of Computer Science and Information Systems in the University of Limerick for their support in completing this project as a part of the MSc programme.

Data Source and Generative AI

I used the ODIR-5K (Ocular Disease Intelligent Recognition) publicly available database for this project, which was provided by Shanggong Medical Technology and Peking University as a part of the ODIR-2019 Grand Challenge. The data can also be found on the Kaggle mirror ([andrewmvd/ocular-disease-recognition-odir5k](https://www.kaggle.com/andrewmvd/ocular-disease-recognition-odir5k)). I did not perform any research activities involving human subjects for this project, and therefore no research ethics application was necessary.

The research design, methods, coding, debugging, interpretation, and critical analysis of results described in this dissertation are my own work. Generative AI tools were used for this project in an auxiliary capacity, primarily to assist with writing code and improving the clarity of text passages that I had already drafted. All content produced with the aid of generative AI tools was reviewed and approved by me.

Table of Contents

Statement.....	i
Abstract	ii
Acknowledgements.....	iii
Data Source and Generative AI	iv
Table of Contents	v
List of Figures	viii
List of Tables.....	ix
Chapter 1: Introduction and Outline	1
1.1 Background and Motivation	1
1.2 Research Objectives	3
1.3 Contributions	4
1.4 Dissertation Outline	5
Chapter 2: Literature Review of Related Work.....	6
2.1 Clinical Context and the Need for Automated Detection	6
2.2 Deep Learning Approaches for Retinal Disease Classification.....	6
2.3 The Interpretability Gap in Medical AI.....	8
2.4 Concept Bottleneck Models	8
2.5 Fuzzy Pattern Trees and Genetic Programming.....	10
2.6 Positioning of RETINEXPLAIN.....	11
Chapter 3: Analytical Background and Requirements Analysis.....	12
3.1 The ODIR-5K Dataset	12
3.2 Functional and Non-Functional Requirements	13
3.3 Resolving the Multi-Label Problem.....	14
3.4 Selection and Validation of the Six Clinical Concepts	15
3.5 Class Imbalance.....	16
3.6 Patient-Level Data Splitting	16

3.7 Choice of Evaluation Baselines	17
Chapter 4: Methodology	18
4.1 Pipeline Overview.....	18
4.2 Stage 1 and 2: Visual Feature Extraction and the Concept Bottleneck Layer	19
4.3 Stage 3: Adaptive Fuzzification	20
4.4 Stage 4: Fuzzy Pattern Tree Representation and Genetic Programming Search..	21
4.5 Fitness Function: From Weighted RMSE to Normalised Soft Macro F1	22
4.6 Baseline Model Configurations	23
4.7 Post-Hoc Interpretability Analysis	24
4.8 Statistical Comparison of Fitness Functions.....	25
Chapter 5: Development	26
5.1 Development Timeline and Key Design Decisions	26
5.2 Dataset Preparation: Implementation.....	26
5.3 Concept Bottleneck Layer: Implementation and Training	27
5.4 Adaptive Fuzzification: Implementation	27
5.5 Fuzzy Pattern Tree Evolution: Implementation	27
5.6 Testing and Debugging	28
5.7 Baseline Model Implementation	29
5.8 Interpretability Analysis Implementation	30
5.9 Statistical Comparison Implementation.....	30
5.10 Technology Stack.....	31
Chapter 6: Results	31
6.1 Dataset and Split Summary.....	31
6.2 Concept Coverage, Correlation and Concept Bottleneck Layer Quality	32
6.3 Fuzzification Behaviour.....	36
6.4 Genetic Programming Evolution and Fitness Function Comparison	37
6.5 Final Classification Performance.....	39

6.6 FPT Tree Size	43
6.7 Feature Usage Frequency Across 30 Runs.....	44
6.8 Readable Branch Expressions for the Best FPT	46
6.9 Logistic Regression Feature Weights and Three-Way Comparison	47
6.10 Statistical Validity	48
6.11 Example Clinical Explanations	49
Chapter 7: Conclusion	50
7.1 Summary of Objectives Met.....	50
7.2 Limitations	51
7.3 Future Work	53
7.4 Closing Remarks	54
References	54
Appendices	57
Appendix A: Concept Keyword Extraction Rules	57
Appendix B: Concept Bottleneck Model Architecture (PyTorch).....	58
Appendix C: Row Normalisation and Soft F1 / Soft MCC / Weighted RMSE Fitness Functions	58
Appendix D: Genetic Programming Toolbox Configuration.....	59
Appendix E: Auxiliary Material Inventory	59

List of Figures

Figure 1: RETINEXPLAIN Four-Stage Pipeline Architecture.....	18
Figure 2: ODIR-5K Class Distribution After Cleaning	32
Figure 3: Concept Label Analysis, Training Set	33
Figure 4: Concept Bottleneck Layer training and validation loss curves.	35
Figure 5: Concept score distributions with P10/P50/P90 percentile markers.....	35
Figure 6: Class-conditional concept score histograms	36
Figure 7: Fuzzy membership function plots overlaid on the empirical data distribution, for each of the seven fuzzified features.	37
Figure 8: Validation macro F1 across all 30 GP runs, best run highlighted.	39
Figure 9: RETINEXPLAIN confusion matrix on the test set (counts and row-normalised percentages).....	41
Figure 10: Per-class precision/recall/F1 bar chart with macro F1 reference line.....	42
Figure 11: FPT best-individual tree size across 30 GP runs, best run highlighted	44
Figure 12: Feature usage frequency across 30 independent GP runs, horizontal bar chart of all 22 fuzzy features.	45
Figure 13: 30-run validation macro F1 distribution comparison (histogram, boxplot, per-run line plot) for Soft F1, Soft MCC and Weighted RMSE	49

List of Tables

Table 1: Fuzzy operators used in the RETINEXPLAIN function set.	21
Table 2: Technology stack.	31
Table 3: Class distribution after cleaning.....	31
Table 4: Concept coverage on the training set (per-image, per-eye).	32
Table 5: Per-concept CBL evaluation on the validation set.	33
Table 6: Concept score percentiles (training set).	34
Table 7: Adaptive fuzzification: best-fit distribution and peak per feature.	36
Table 8: Fuzzy Pattern Tree results by fitness function (test set).	38
Table 9: Final model comparison (test set).	39
Table 10: RETINEXPLAIN (soft macro F1) per-class performance on the test set.....	42
Table 11: FPT tree size across 30 runs, by fitness function.	43
Table 12: Concept-level feature usage across 30 runs versus CBL AUC (soft macro F1 configuration).	44
Table 13: Best FPT (Run 21) per-class branch expressions.....	46
Table 14: Selected logistic regression weights (raw-feature model), by class and its clinically associated concept.	48
Table 15: 30-run validation macro F1 distributions, by fitness function.	48
Table 16: Example correctly classified test-set predictions with detected clinical signs.....	50

Chapter 1: Introduction and Outline

1.1 Background and Motivation

Retinal diseases have become one of the leading causes of untreatable and inevitable blindness in the world. Diabetic retinopathy, glaucoma, age-related macular degeneration (AMD), cataract, hypertensive retinopathy and myopic degeneration combined, account for a significant proportion of vision loss worldwide, while the WHO has estimated nearly 2.2 billion people to have some form of vision impairment, of which a considerable proportion is detectable via retina examination (World Health Organisation, 2019). Retinal fundus photography, a non-invasive and inexpensive screening procedure, in which a camera captures an image of the interior of the eye, has become the cornerstone of automated retinal disease screening research.

Deep learning and particularly convolutional neural networks (CNNs) have seen tremendous success in the domain in the last decade. With the advent of transfer learning from ImageNet pretrained models such as EfficientNet, DenseNet, ResNet or InceptionNet, multiple works have shown that fine-tuning CNNs on retinal scans allows training diagnostic-level models for several classes of diseases detectable on the retina, which perform on par with human-level experts.

Such successes are grounded in a combination of higher quality data, better transfer-learning techniques, and better model architectures. Nevertheless, there is one major caveat with these kinds of CNN-based models: while they often perform very well on diagnostic and screening tasks, they are ultimately black-box models. A CNN fine-tuned on disease labels to classify retinal scans is ultimately a black-box model: while the internal representation it learns for the image is very often useful for downstream tasks, it is not accessible and cannot be interpreted by humans. This becomes a problem when one needs to explain the reasoning behind the model's decision to clinicians, or even patients, which is frequently necessary in medical settings. The industry standard for such cases is to apply post-hoc explanation algorithms such as Grad-CAM or any of the existing implementations of the LIME or SHAP frameworks, which attempt to approximate the reasoning of the CNN classifier from the "inside."

The value of such interpretability methods is hard to overestimate. At the same time, all of these methods are ultimately only approximations of how a CNN may reason about an image, and offer little guarantee regarding the connection between the input image and the final classification. Therefore, they cannot be reliably used in critical applications such as medical

imaging analysis, where errors due to misinterpretation simply cannot be tolerated. This shortcoming becomes especially pertinent with the new regulation changes in the EU: with the AI Act now in force since August 2024, medical diagnostic software falls under the classification of “high risk,” and Article 86 of the Act requires that a user affected by an AI decision (such as a patient diagnosed by an AI medical imaging system) should be able to get a satisfactory explanation of how the conclusion was reached. An AI model that reliably classifies retinal scans into diagnostic categories but cannot explain its reasoning is no longer compliant with the latest regulations.

The present dissertation serves to fill this gap: it proposes RETINEXPLAIN , a concept bottleneck model, which offers a middle ground between purely black-box (“explained later”) and fully interpretable but often shallow (“build it yourself”) approaches. Two recent research directions have proposed intriguing possibilities for accomplishing similar goals. The first, concept bottleneck models (CBMs) of Koh et al. (2020), imposes constraints on the model architecture to make it inherently interpretable: when classifying a fundus image into diagnostic categories, such a model would first predict concept labels (textual descriptions) or standardised labels from an ontology, which would then be used as the basis for generating the final diagnosis. This allows for post-hoc interpretability: a human interpreter would be able to see what concepts the model recognised in the image as the basis for its diagnostic decision.

The other promising avenue of research, described in de Lima et al. (2024, 2025), concerns the creation of Fuzzy Pattern Trees via Genetic Programming (GP). The method generates human-interpretable fuzzy diagnostic trees: if a patient’s age and sex, along with the characteristics of their retina scan, satisfy certain fuzzy conditions, they may be diagnosed with one of the seven diseases that RETINEXPLAIN classifies. This approach serves as the foundation for building fully interpretable CNNs, in which each decision is made following an interpretable set of rules.

RETINEXPLAIN employs many concepts from both research directions to create an explainable AI pipeline for retinal disease classification. It is a deep learning-based diagnostic pipeline for multi-class classification of retinal diseases with an explainable AI component which follows a four-stage processing chain. The first stage employs an EfficientNet-B0 pretrained on ImageNet weights as a frozen backbone, which extracts features from a fundus image. In the second stage, a small, trainable Concept Bottleneck layer combines concepts to predict the presence of disease. In the third stage, these features along with the patient’s age and sex are fuzzified with an adaptive fuzzification strategy and then used to train the Fuzzy

Pattern Trees with a Winner-Takes-All root node and seven branches in total, evolved with GP, which produce the final diagnostic class for the image.

1.2 Research Objectives

This dissertation addresses the following research question:

To what extent can a GP-evolved Fuzzy Pattern Tree operating on concept-bottleneck features match the classification performance of comparable interpretable and black-box models on seven-class ODIR-5K, and what is the measurable cost in macro F1 of the interpretability constraint?

This question is motivated by a gap in the existing FPT literature: prior work on GP-evolved Fuzzy Pattern Trees has operated exclusively on hand-crafted tabular features, and no prior study has tested whether FPTs remain competitive when their input features are derived from a deep visual backbone through a concept bottleneck, rather than supplied directly. RETINEXPLAIN is, to the best of the author's knowledge, the first system to do so on a real-world medical imaging benchmark, and the answer to this question has implications beyond ODIR-5K for any domain where interpretability is required but input features must be learned from raw data rather than hand-crafted.

This work aimed to accomplish the following objectives:

1. Construct an end-to-end pipeline comprising a deep visual feature extractor, a Concept Bottleneck Layer, a fuzzy representation of features, and a Genetic Programming-evolved Fuzzy Pattern Tree classifier to perform multi-class classification on the ODIR-5K dataset for the task of retinal disease detection.
2. Derive the concept labels needed to train the aforementioned Concept Bottleneck Layer from the diagnostic keyword annotations already present in the ODIR-5K dataset, thus bypassing the computationally and temporally expensive process of obtaining them through manual labelling by human experts, while simultaneously assessing whether these six concepts provide satisfactory coverage and are not overly redundant in expressing the diagnostic information.
3. Apply an adaptive fuzzification scheme, in which the support of each fuzzy membership function is learned for each feature based on its empirical distribution, rather than being fixed to a uniform interval, as proposed in de Lima et al. (2025)

4. Compare the performance of the Fuzzy Pattern Tree obtained through Genetic Programming against a variety of different methods, ranging from highly interpretable ones, such as logistic regression and decision trees, through more competitive but less interpretable ensemble methods, such as random forests, and ultimately the task of fine-tuning an EfficientNet-B0 classifier through end-to-end training, in order to obtain an upper bound on performance for the given task.
5. Report the results in terms of macro F1, highlighting the cases in which the manually interpretable model failed to compete with the performance of the other approaches, as well as critically assessing the limitations of the concept derivation procedure, in particular that several concepts function as near-deterministic proxies for their associated disease labels rather than independently learned visual signs.

1.3 Contributions

This dissertation makes the following contributions.

- A working four-stage pipeline combining deep feature extraction, clinical concept supervision, adaptive fuzzy logic and Genetic Programming into a single explainable classifier for a genuinely difficult seven-class medical imaging problem, where a comparable black-box CNN reaches only around sixty percent accuracy (Mostafa et al., 2023; confirmed independently by this project's own EfficientNet-B0 baseline).
- An automatic concept labelling method that derives clinical sign supervision from ODIR-5K's free-text diagnostic annotations, validated through a concept coverage and correlation analysis rather than asserted without evidence.
- An application of the adaptive, distribution-fitted fuzzification method of de Lima et al. (2025) to a new domain, showing that Concept Bottleneck Layer scores are strongly right-skewed in this setting, making adaptive fuzzification materially more appropriate than the conventional uniform scheme.
- A broad comparison across the interpretability spectrum (logistic regression, decision tree, Fuzzy Pattern Tree, random forest, and an end-to-end black box), showing the evolved Fuzzy Pattern Tree achieves a macro F1 effectively tied in practical terms with the strongest interpretable baseline, with a difference of

0.0010 macro F1 well within the run-to-run variance of RETINEXPLAIN's own 30 independent GP runs, while substantially exceeding it on accuracy, corroborated by a three-way convergence of evidence (logistic regression weights, CBL detection accuracy, and evolved-tree feature usage) around clinically established concept-disease associations.

- A statistically supported justification for the choice of fitness function: a paired significance test across thirty independent runs of three candidate fitness functions, rather than an informal preference.

1.4 Dissertation Outline

The remainder of this dissertation is organised as follows.

Chapter 2 reviews the relevant literature across four areas: deep learning approaches to retinal disease classification; the interpretability gap in medical AI and the regulatory context that motivates closing it; Concept Bottleneck Models as an intrinsically interpretable architecture; and Fuzzy Pattern Trees evolved by Genetic Programming as a white-box classification method, including the specific fitness function and fuzzification innovations from de Lima et al. (2024, 2025) that RETINEXPLAIN builds directly on.

Chapter 3 provides the analytical background to the project: a description of the ODIR-5K dataset and its structure, the requirements the system needed to satisfy (both functional, such as producing seven-class predictions, and non-functional, such as leak-free patient-level evaluation), and the key design decisions that were made before implementation began, including how the multi-label structure of the dataset was resolved.

Chapter 4 sets out the methodology: the overall four-stage architecture, the mathematical formulation of each stage (the Concept Bottleneck Layer, the adaptive fuzzification scheme, the Fuzzy Pattern Tree representation and the Genetic Programming search), and the fitness functions evaluated, including the evolution from an automatically weighted RMSE loss to a row-normalised soft macro F1 loss and the reasoning behind that change.

Chapter 5 describes the development of the system: the specification and design of each pipeline stage, the implementation details (dataset preparation, model architectures, training configuration, GP toolbox setup), and the testing and debugging process, including four significant bugs encountered during development and how each was identified and resolved.

Chapter 6 presents the results. This includes the concept coverage and correlation analysis, the per-concept quality of the trained Concept Bottleneck Layer, the behaviour of the adaptive fuzzification scheme, the evolutionary convergence of the Genetic Programming search, and the final classification performance of the Fuzzy Pattern Tree compared against the logistic regression and EfficientNet-B0 baselines, with critical analysis of what these results do and do not show.

Chapter 7 concludes the dissertation, summarising how the stated objectives were met, discussing the limitations of the work, and setting out concrete directions for future work, including several that were explicitly deferred during the project due to time constraints (most notably automated rule extraction).

References and appendices follow, with the appendices containing supplementary material including selected code listings and additional figures referenced from the project notebooks.

Chapter 2: Literature Review of Related Work

2.1 Clinical Context and the Need for Automated Detection

Diabetic retinopathy, glaucoma, age-related macular degeneration, cataract and hypertensive retinopathy are the major causes of preventable blindness in the world. The importance of addressing this problem cannot be overstated, mainly because delays in getting the right treatment can lead to permanent vision loss for people with many diseases (Muchuchuti and Viriri, 2023). Given this, retinal fundus photography is particularly significant as it is a safe and relatively simple procedure compared to other diagnostic tests. It is also relatively affordable and quick, which makes it highly suited to large-scale automated screening, dominating the literature on the classification of retinal diseases.

2.2 Deep Learning Approaches for Retinal Disease Classification

Convolutional Neural Networks have been widely adopted for automated retinal disease classification over the last decade. The use of CNNs for automated classification requires no manual identification of image descriptors since typical CNNs can directly learn visual features from raw images. Moreover, transfer learning from large natural image datasets such as ImageNet has proven particularly effective for medical image analysis, due to their limited availability of annotated training data (Tan and Le, 2019). Many architectures have been used

for retinal disease classification, including EfficientNet, DenseNet, MobileNet, ResNet, and InceptionNet. EfficientNet is highlighted by the authors as a particularly promising architecture, due to its compound scaling coefficient, which allows it to achieve comparable performance to other networks with fewer parameters.

A thorough review of the literature conducted by Muchuchuti and Viriri (2023) on deep learning applications to retina disease detection demonstrates that Convolutional Neural Networks and Vision Transformers demonstrate high diagnostic accuracy in diabetic retinopathy, AMD, glaucoma, multi-disease classification tasks, and benefit from ensemble strategies and attention mechanisms. The ODIR-5K dataset used in the current project, which contains colour fundus images from 5,000 patients diagnosed with one of the eight disease categories, is discussed in said review as a common benchmark used in multi-classification studies, and has seen its share of research published.

Hussein, Farouk and Saeid (2025) describe their findings from exploring one retina disease dataset, combining DenseNet121 and MobileNetV2 feature extraction and Principal Component Analysis for dimensionality reduction, claiming a peak of 98.2% multi-classification accuracy (not macro F1). However, the same study observes a stark decrease in performance (98.2% $\rightarrow$ 74%) on a different test set that the model was not trained on, which is an issue of generalizability that the current project hopefully does not have to deal with.

Most relevant to the current project, however, is the study conducted by Mostafa et al. (2023), which touches upon the difficulties of multi-classification (more than two classes) in the ODIR dataset (the current project uses the same classification framework, excluding the “Other” category for reasons discussed in Chapter 3) claiming less than 61% accuracy in multi-class CNNs for the same task, due to the lack of consistency between the images, the diagnosis, and the patients’ characteristics, while achieving up to 98 – 100% accuracy in binary versions of the same task. This is a critical piece of information for interpreting the results of Chapter 6, which discusses the performance of the EfficientNet-B0 baseline trained as part of the current project: 60.12% accuracy on the seven-class ODIR classification task, similar to the findings of Mostafa et al. (2023) - demonstrating that the task itself is non-trivial, and a lower than 70% accuracy is not a consequence of the model’s design and training procedure being somehow “suboptimal” compared to other approaches.

2.3 The Interpretability Gap in Medical AI

Although this classification performance was impressive, most published retinal disease detection systems are built using deep models that act as black boxes, post-hoc explanations, such as Grad-CAM (Selvaraju et al., 2017), LIME (Ribeiro, Singh and Guestrin, 2016) or SHAP (Lundberg and Lee, 2017), provided for interpretability. While these post hoc explanation techniques attempt to recreate an approximated version of the model's reasoning process, the resulting explanation may bear little to no resemblance to what actually occurred in the individual model instance. Muchuchuti and Viriri (2023) note that this presents one of the unresolved challenges in the field, citing a general lack of interpretability, with the explanations being non-trivial to understand for either the clinician reviewing the case or the patient receiving treatment. Neural networks, in general, are notoriously difficult to interpret.

This presents a significant problem, as regulatory agencies such as the EU's AI Act (European Parliament and Council of the European Union, 2024) or the FDA's Software as a Medical Device guidance, effectively, require explainable AI for such applications. Specifically, Article 13 of the EU AI Act effectively requires that the high-risk AI systems, which medical diagnostic models are explicitly considered, should be transparent enough that their decisions could be understood and reviewed by a human being. Additionally, per Article 86 of the same act, the affected party has a right to an explanation in case an organization uses an AI system to make a significant decision impacting them, directly or indirectly. In short, while an AI classifier with an accuracy of 99% may sound desirable, it has very little use in practice if it is impossible to explain to either the doctor reviewing the case or the patient undergoing treatment, which factors the algorithm reviewed in making its suggestion. As such, it becomes imperative to design a model that has explainability built in, rather than attempting to retrofit explanation mechanisms onto the existing model architecture.

2.4 Concept Bottleneck Models

Concept Bottleneck Models as introduced by Koh et al. (2020) take a step towards intrinsically interpretable deep learning for classification tasks. In contrast to standard models that directly learn the mapping from pixels to a label, a Concept Bottleneck Model (CBM) factorizes the learning process into two steps. First, the input is encoded into a set of human-readable concepts and subsequently, these concept scores are used to make a prediction. This design has two advantages in terms of interpretability. Firstly, the learned concept representations in the intermediate layer are by construction human-interpretable: each neuron in this layer encodes

the relevance of a specific concept (as defined by the researchers) for the prediction. Secondly, the CBM allows for test-time interventions as described by Koh et al.: during inference, a human analyst can directly intervene in the predicted concept values (e.g. correct a misunderstood concept) and trace the effect of this intervention on the final model prediction.

When it comes to retina diseases, the set of concepts naturally consists of the disease-specific visual biomarkers clinicians look out for when interpreting a fundus image. For instance, haemorrhages and exudates are typical signs of diabetic retinopathy, drusen are indicative of age-related macular degeneration, while disc pallor and cupping are seen in glaucoma patients. Lastly, tortuous blood vessels point to hypertensive retinopathy. Niemeijer et al. (2007) and Annunziata et al. (2016) provide detailed clinical evidence linking these fundus imaging biomarkers to their respective diseases. Specifically, RETINEXPLAIN uses six concepts: haemorrhage, drusen, disc pallor, vessel tortuosity, red spot and exudate, which have all been identified as disease-specific biomarkers based on the medical literature.

It must be stressed that, as elaborated upon further in Chapters 3, 5 and 6 of this dissertation, the way concept annotations were obtained in this project differs in one important way from the ideal setup outlined by Koh et al. (2020): the latter suggest acquiring concept labels via separate independent expert annotations for each image, while for ODIR-5K, concepts were extracted from the keyword tags the experts chose to report when classifying an image. As explained in detail in Section 3.4 and Section 5.2, six visual biomarkers were extracted from the keyword annotation set of the ODIR-5K dataset. This choice was motivated by the practical consideration of not having any other source of annotated data. However, this decision comes at the cost of conceptual dependency: three of the six concepts (disc pallor, drusen, vessel tortuosity) are effectively proxies for the three Glaucoma, AMD and Hypertension labels, as the keywords indicating these biomarkers only occur in the diagnostic notes for the corresponding classes (see also Section 6.2 for discussion). The other three concepts (haemorrhage, red spot, exudate) are relatively independent as there is a variety of different descriptive phrases the annotators could have used. This nuance is especially relevant for this project, as much of Section 6.2 revolves around testing the effects of this particular methodological choice, which is why the concepts in RETINEXPLAIN should be discussed as proxies for diseases based on the keyword annotation methodology.

2.5 Fuzzy Pattern Trees and Genetic Programming

Fuzzy Pattern Trees are a tree-structured, symbolic classification model in which internal nodes are fuzzy logical operators (such as minimum, maximum, weighted average, ordered weighted average, dilator, concentrator and complement) and leaf nodes are fuzzy features, each of which is the degree of membership of an input value in a named fuzzy set. Applying the operators in sequence, from leaves to root, propagates membership values upward and produces, at the root, a score in the interval $[0, 1]$ representing the tree's confidence that the input belongs to a given class. FPTs originate with the pattern tree formulation of Huang and Gedeon (2006) and Huang et al. (2008), which distinguished pattern trees from earlier fuzzy decision trees by generating one tree per output class rather than splitting samples by class, and by permitting a richer set of aggregation operators than the minimum and maximum operators used in fuzzy decision trees. Senge and Hüllermeier (2011) subsequently introduced a top-down induction method as an alternative to the original bottom-up approach, extending the range of usable operators.

The critical development for this dissertation is the application of Genetic Programming to the induction of FPTs, which allows the tree structure itself to be searched for automatically rather than induced by a greedy heuristic. Koza's foundational work on Genetic Programming (Koza, 1992) established the representation of candidate programs as tree structures and the evolutionary process, based on tournament selection, crossover and mutation, used to search the space of such programs across successive generations. Early applications of evolutionary search to FPT induction used Cartesian Genetic Programming (dos Santos and Amaral, 2015) and later Grammatical Evolution (Murphy et al., 2020, 2022), both of which demonstrated that evolved FPTs could achieve accuracy competitive with black-box models such as Random Forests while retaining full interpretability.

The GP framework adopted by RETINEXPLAIN is that of de Lima et al. (2024), "Fuzzy Pattern Trees for Classification Problems Using Genetic Programming", which uses standard Koza-style Genetic Programming (rather than Cartesian GP or Grammatical Evolution) to evolve FPTs, and represents a multi-class problem using a single tree whose root is a Winner-Takes-All node with one branch per class. de Lima et al. evaluate candidate trees using a Root Mean Squared Error fitness function computed between the tree's branch outputs and a one-hot encoding of the true class, rather than treating the problem as a simple 0/1 classification loss. This last point matters: an aggregate error measure computed against a soft, one-hot target rewards a tree not merely for predicting the correct class but for being confidently correct, which de Lima et al. show provides more useful gradient-like signal to the evolutionary search

than a hard classification loss would. Their paper additionally introduces and benchmarks several parsimony pressure methods, including lexicographic parsimony pressure embedded in Lexicase Selection (Lexi2) and batch variants thereof, to control tree bloat and encourage smaller, more interpretable solutions, and reports that plain tournament selection remained the most accurate selection method overall across the 20 benchmark datasets tested, with Lexicase-based approaches offering the best size-versus-accuracy tradeoff. RETINEXPLAIN adopts the same GP framework, WTA structure and tournament selection strategy from this paper, but replaces the RMSE fitness function with a normalised Soft Macro F1 loss; the motivation for this choice is detailed in Chapter 4.

A second, closely related paper, de Lima et al. (2025), "On fitting numerical features into probabilistic distributions to represent data for fuzzy pattern trees", is the direct source of the fuzzification method used in RETINEXPLAIN. That paper observes that the fuzzification schemes conventionally used in the FPT literature, most commonly three uniformly spaced triangular membership functions spanning the observed minimum to maximum of a feature, implicitly assume the underlying feature is roughly uniformly, normally or triangularly distributed around the centre of its domain. Where this assumption does not hold, which the paper demonstrates is common in real-world datasets, the resulting fuzzy terms ("low", "medium", "high") no longer correspond in any meaningful sense to genuinely low, medium or high values of the underlying feature, undermining exactly the interpretability that fuzzification is meant to provide. de Lima et al. (2025) propose fitting each feature automatically to one of three candidate distributions (triangular, exponential, uniform) by maximum log-likelihood, and setting the fuzzy membership function bounds according to the best-fitting distribution's characteristic point (its peak, for a triangular fit, or its mean, for an exponential fit) rather than the naive midpoint of the feature's observed range. Their experiments, across eleven benchmark classification problems and six parsimony pressure methods, show that this adapted scheme improves the accuracy-versus-size Pareto front more often than the conventional uniform scheme, and, notably for this dissertation, more often than a Gaussian membership function alternative that they also tested. RETINEXPLAIN applies this method directly, as described in Chapter 4.

2.6 Positioning of RETINEXPLAIN

RETINEXPLAIN occupies a position in the literature that, to the best of my knowledge based on the review above, has not previously been explored. Existing interpretable approaches to

retinal disease detection either apply post-hoc explanation methods to an otherwise black-box CNN, which does not guarantee a faithful explanation of the model's actual reasoning, or apply a shallow, inherently interpretable model (such as a decision tree or a rule-based system) directly to hand-crafted features, forgoing the representational power of a learned deep visual feature extractor entirely. RETINEXPLAIN combines both: a deep, pretrained visual backbone supplies rich image features; a Concept Bottleneck Layer compresses those features into named clinical concepts, restoring interpretability at that stage; adaptive fuzzification, following de Lima et al. (2025), converts those concepts into fuzzy linguistic terms appropriate to their actual empirical distribution; and a Genetic Programming-evolved Fuzzy Pattern Tree, following de Lima et al. (2024), classifies the result using a fully symbolic, human-traceable expression. Every prediction the system produces can, in principle, be traced from the raw image through named concepts to an explicit fuzzy rule, addressing the interpretability gap identified in Section 2.3 by construction rather than by post-hoc approximation, while the difficulty of the seven-class ODIR-5K benchmark, established independently by Mostafa et al. (2023) and reproduced by this project's own black-box baseline, situates the accuracy achieved by RETINEXPLAIN's interpretable classifier in a genuinely demanding setting rather than an artificially easy one.

Chapter 3: Analytical Background and Requirements Analysis

3.1 The ODIR-5K Dataset

ODIR-5K (Ocular Disease Intelligent Recognition) is a publicly available dataset released by Shangong Medical Technology and Peking University as part of the ODIR-2019 Grand Challenge. It contains fundus photographs and associated clinical records collected from patients across a number of hospitals in China, and was accessed for this project via its Kaggle mirror, which distributes it as a single `full_df.csv` file together with a folder of preprocessed images.

Each row of the underlying dataset corresponds to one retinal fundus image (one eye of one patient), and records the patient's unique identifier, age, sex, the free-text diagnostic keywords recorded for the left eye and for the right eye of that patient, and a labels field giving the single-letter or multi-letter disease code(s) assigned to that specific image. The eight possible disease codes in ODIR-5K are: N (Normal), D (Diabetic Retinopathy), G (Glaucoma), C (Cataract), A (Age-related Macular Degeneration), H (Hypertension), M (Myopia) and O (Other). This

project uses only the seven primary disease codes, excluding O, for reasons set out in Section 3.3.

Two structural features of the raw dataset required explicit analytical attention before any modelling could begin. First, each patient contributes two images (left eye and right eye), and both eyes typically reflect a shared underlying disease state; treating the two eyes of a single patient as fully independent samples, while common in the retinal imaging literature as a means of doubling the effective sample size, risks leaking information between training and evaluation sets if a naive image-level split is used, since both eyes could plausibly end up in different splits despite being highly correlated. This was identified explicitly at the start of the project (see Section 5.1) and resolved by splitting the dataset at the patient level, not the image level, described in Section 3.6.

Second, and less immediately obvious, a patient can be recorded with more than one disease label if their diagnostic notes mention more than one condition. This means ODIR-5K is, in its native form, a multi-label rather than a single-label classification problem, which is incompatible with the Winner-Takes-All, single-tree-per-class formulation of Fuzzy Pattern Trees used in de Lima et al. (2024) and adopted for this project, and its resolution is described in Section 3.3.

3.2 Functional and Non-Functional Requirements

The following requirements were established, primarily through consultation with the project supervisors during the early weeks of the project, before implementation of the core pipeline began.

Functional requirements.

- The system must classify a retinal fundus image, together with the associated patient age and sex, into one of seven disease categories: Normal, Diabetic Retinopathy, Glaucoma, Cataract, AMD, Hypertension or Myopia.
- The system must produce, for every prediction, an intermediate representation of six named clinical concepts (haemorrhage, drusen, disc pallor, vessel tortuosity, red spot, exudate), each expressed as a degree of membership rather than a simple present/absent flag, so that the basis for a given prediction can be inspected.
- The final classification stage must be a symbolic, human-traceable model (a Fuzzy Pattern Tree) rather than an opaque model, so that the path from concept scores to

final prediction can, in principle, be read as an explicit rule rather than approximated by a post-hoc method.

- Concept labels used to train the Concept Bottleneck Layer must be derivable automatically from information already present in ODIR-5K (the diagnostic keyword text), without requiring new manual annotation by a clinical expert, since no such annotation resource was available within the scope or timeline of this project.

Non-functional requirements.

- Evaluation must be leak-free: no patient may appear in more than one of the training, validation and test splits, given that both eyes of a single patient are correlated (Section 3.1).
- The pipeline must be reproducible: dataset splits, model initialisation and the Genetic Programming search must all be seeded, so that results can be regenerated and compared across pipeline revisions during development.

3.3 Resolving the Multi-Label Problem

Because a patient's diagnostic keywords can mention more than one condition, the raw ODIR-5K dataset is multi-label: a given image's labels field can contain more than one disease code (for example, a patient with both diabetic retinopathy and hypertension). A single-label classifier, including the Winner-Takes-All Fuzzy Pattern Tree formulation this project relies on, cannot be trained directly on multi-label data without some form of conversion.

Three candidate conversion strategies were considered, with the decision based on evidence from the dataset itself rather than an untested assumption.

1. **Clinical severity priority.** Each patient with multiple labels is assigned the single most sight-threatening condition present, using a fixed clinical priority ordering (Diabetic Retinopathy, then Glaucoma, AMD, Hypertension, Cataract, Myopia).
2. **Most frequent in dataset.** Each patient with multiple labels is assigned whichever of their labels is more prevalent across the dataset as a whole.
3. **Drop multi-label rows.** Patients with more than one disease label are simply excluded from the dataset, leaving only patients with an unambiguous single diagnosis.

All three strategies were implemented and compared directly on the dataset (after removing the "Other" category, discussed below, in every case). Strategies 1 and 2 produced almost identical

resulting class distributions to one another (for example, 1,626 versus 1,608 Diabetic Retinopathy images and 76 versus 72 Hypertension images respectively), and strategy 3 produced a somewhat smaller but proportionally very similar distribution to strategies 1 and 2. Because the resulting class balance was essentially unaffected by the choice of strategy, and because dropping multi-label rows is the simplest of the three approaches to justify and describe precisely in a dissertation (it requires no priority ordering to defend and introduces no risk of a clinically debatable tie-breaking rule), strategy 3 was adopted.

A more direct resolution of the multi-label issue was provided by the ODIR-5K Kaggle distribution itself, which includes a native per-image labels column assigning a single diagnosis to each image based on the diagnostic keywords of the corresponding eye. This avoids the need to combine patient-level disease indicators and subsequently resolve multi-label cases through a post-hoc rule. The native per-image label was therefore adopted for the final cleaned dataset used throughout the project.

The eighth ODIR-5K category, "Other", covering a heterogeneous set of miscellaneous conditions not otherwise classified, is excluded throughout this project. This category was identified during dataset inspection as the single largest source of multi-label combinations (a large proportion of multi-label cases involve one label from the seven principal disease codes and a co-occurring "Other" label), and, being a catch-all category with no single coherent clinical definition, it is not a meaningful target class for a Concept Bottleneck Layer trained on six specific, named clinical signs.

3.4 Selection and Validation of the Six Clinical Concepts

The six clinical concepts used in RETINEXPLAIN (haemorrhage, drusen, disc pallor, vessel tortuosity, red spot, exudate) were selected on the basis that they are the principal visual biomarkers associated with the seven target disease classes in the established ophthalmology literature: haemorrhages and exudates are canonical signs of diabetic retinopathy; drusen are the hallmark of AMD; disc pallor is associated with glaucomatous optic nerve damage; vessel tortuosity is a recognised marker of hypertensive retinopathy; and red spot is an additional visual sign of diabetic retinopathy. A small number of papers reviewed also mention cotton wool spots and neovascularisation as additional signs of diabetic retinopathy, but these were judged to be advanced-stage indicators outside the scope of this project and were not added to the concept set.

Beyond the literature-based justification alone, each concept's inclusion was validated empirically through a concept coverage and correlation analysis, performed on the training split after the dataset cleaning described in Section 3.3. Concept coverage measures what proportion of training images contain each concept's associated keyword phrases in their diagnostic text (Section 6.2 reports this analysis in full). Concept correlation measures whether any two concepts co-occur so frequently in the training data that they might be functionally redundant. Both analyses were performed after switching to the final per-image, per-eye labels (Section 5.2), since prior to that the dataset was structured at patient level, meaning any coverage figures would combine keyword information from both eyes into a single row and would consequently not be a reliable measure of per-image concept prevalence.

3.5 Class Imbalance

The cleaned, single-label ODIR-5K dataset used in this project (5,684 images after adopting the per-image labels column, Section 3.3) is substantially imbalanced. Normal and Diabetic Retinopathy together account for close to 79% of all images (50.5% and 28.3% respectively), while the remaining five classes range from 5.2% (Cataract) down to 2.3% (Hypertension, 128 images in total). This imbalance has consequences at every stage of the pipeline: it required positive class weighting when training the Concept Bottleneck Layer (Section 5.3), it motivated the use of macro F1 as the primary comparison metric rather than accuracy, and it is the leading candidate explanation, discussed critically in Section 6.5, for the comparatively weak performance of the Fuzzy Pattern Tree on the smallest classes (Hypertension and Myopia in particular).

3.6 Patient-Level Data Splitting

As set out in Section 3.1, splitting the dataset at the image level risks leaking information between training and evaluation, since both eyes of a single patient share a common disease state and are drawn from the same underlying diagnostic examination. RETINEXPLAIN therefore splits the dataset at the patient level throughout: each unique patient ID is assigned to exactly one of the training, validation or test sets, and every image belonging to that patient (both eyes, where both are present and pass the single-label filtering of Section 3.3) is assigned to that same split. The split is performed in two stages using scikit-learn's `train_test_split`, first separating 70% of patients into a training set and holding out the remaining 30%, then splitting that remaining 30% evenly into a 15% validation set and a 15% test set, with stratification by disease label applied at each stage to keep the class distribution broadly consistent across splits.

The same seed is used throughout, and an explicit overlap check (reported in Section 6.1) confirms zero patient ID overlap between any pair of splits in the final pipeline.

3.7 Choice of Evaluation Baselines

A broad set of four baselines was established to contextualise the performance of the Fuzzy Pattern Tree classifier, spanning the full interpretability spectrum from a simple linear model through to a fully opaque deep network trained with no interpretability constraint at all.

The logistic regression baseline is trained on exactly the same feature representations available to the Fuzzy Pattern Tree, and is run in three configurations: a demographic-only floor using patient age and sex alone; the raw Concept Bottleneck Layer scores together with age and sex; and the full fuzzified 22-dimensional feature representation. All three are reported in Chapter 6, since together they show clearly how much each stage of the pipeline (concept extraction, then fuzzification) contributes.

The decision tree and random forest baselines are each run on two feature sets, the raw concept-plus-demographic features and the 22 fuzzified features, with fine-tuned hyperparameters (Section 5.7). The decision tree is a fair interpretable rival to the Fuzzy Pattern Tree: both produce a symbolic structure a person could read directly, but a decision tree partitions the feature space with hard splits, whereas an FPT combines fuzzy membership degrees through continuous operators, so comparing the two tests whether the additional complexity of the fuzzy representation and evolutionary search earns a genuine accuracy advantage over a simpler, off-the-shelf interpretable model. The random forest is not interpretable, but as an ensemble of many decision trees it typically recovers much of the accuracy available in a given feature set, so it serves as a practical upper bound on what a non-interpretable model can extract from the same 22 fuzzy or 8 raw features RETINEXPLAIN itself uses.

The EfficientNet-B0 end-to-end baseline fine-tunes the same backbone architecture used inside RETINEXPLAIN's own Concept Bottleneck Layer but trains it directly and fully on the seven disease labels, with no concept bottleneck and no interpretability constraint of any kind. It establishes the accuracy a comparable black-box model can achieve on this exact task from the raw image alone, with none of RETINEXPLAIN's interpretability machinery in the way. The gap between it and RETINEXPLAIN's Fuzzy Pattern Tree is presented in Chapter 6 as the outer bound of the measurable accuracy cost of interpretability, while the gap to the random forest

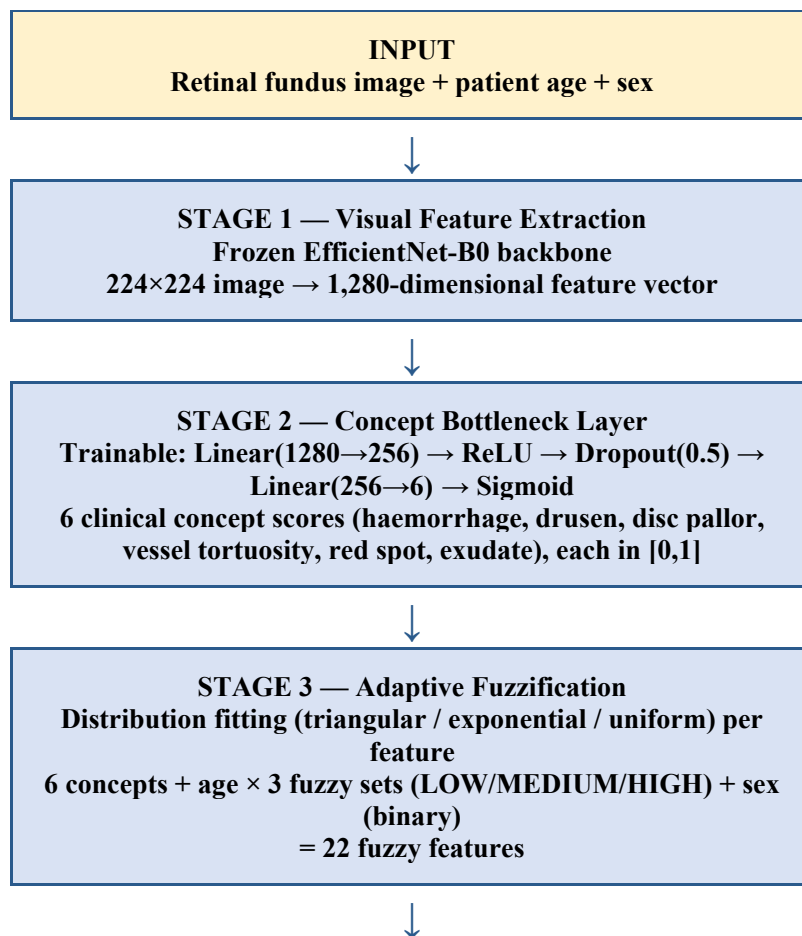
baseline is presented as a tighter, fairer estimate of that cost, since the random forest at least uses the same concept-derived features RETINEXPLAIN itself is built on.

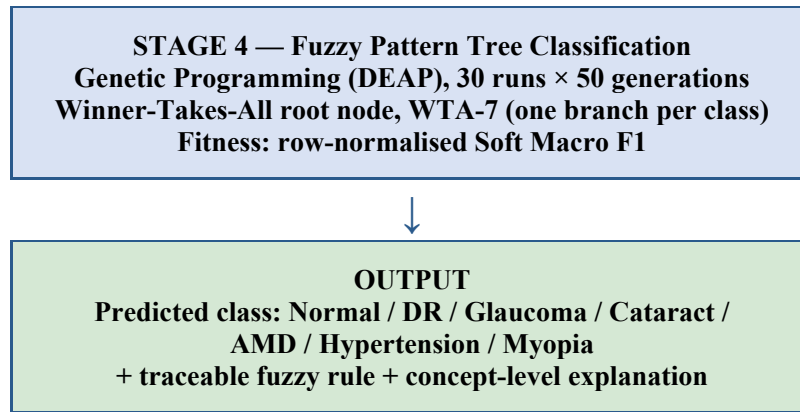
Chapter 4: Methodology

4.1 Pipeline Overview

RETINEXPLAIN is structured as four sequential stages, illustrated in Figure 1. A frozen EfficientNet-B0 backbone extracts a 1,280-dimensional visual feature vector from each retinal fundus image (Stage 1). A trainable Concept Bottleneck Layer maps that feature vector to six clinical concept scores, each in the interval $[0, 1]$ (Stage 2). Those six scores, together with patient age and a binary sex indicator, are converted into 22 fuzzy membership values using an adaptive triangular fuzzification scheme (Stage 3). Finally, a Fuzzy Pattern Tree, evolved by Genetic Programming under a seven-branch Winner-Takes-All root node, consumes those 22 fuzzy features and produces the final disease classification (Stage 4). Each stage is described mathematically below; implementation details are given in Chapter 5.

Figure 1: RETINEXPLAIN Four-Stage Pipeline Architecture





4.2 Stage 1 and 2: Visual Feature Extraction and the Concept Bottleneck Layer

Let x denote a retinal fundus image after preprocessing. A pretrained EfficientNet-B0 backbone f , with weights frozen throughout training, maps x to a 1,280-dimensional feature vector v . The backbone weights are never updated; it is used purely as a fixed visual descriptor, following the standard transfer learning practice of using an ImageNet-pretrained network as a feature extractor when the downstream labelled dataset is comparatively small, which ODIR-5K is relative to ImageNet.

A trainable Concept Bottleneck Layer maps v to six raw concept logits, which are passed through a sigmoid function to produce six concept scores, each in the range $[0, 1]$, one score per clinical concept (haemorrhage, drusen, disc pallor, vessel tortuosity, red spot, exudate). The concept layer consists of a flatten operation, a linear layer from 1,280 to 256 units, a ReLU non-linearity, a dropout layer with rate 0.5 to reduce overfitting, and a final linear layer from 256 to 6 units.

The Concept Bottleneck Layer is trained by minimising a weighted binary cross-entropy loss, summed across all six concepts. For each concept k , the loss applies a positive class weight to compensate for concept rarity, computed as the ratio of negative to positive training examples for that concept and capped at a maximum of 10 to prevent any single rare concept from dominating the loss. Training uses the Adam optimiser with a learning rate of 0.0001 and weight decay of $1e-4$, for up to 50 epochs with early stopping monitored on validation loss. In the final training run reported in Chapter 6, the best validation loss was reached at epoch 35 and training stopped at epoch 42.

4.3 Stage 3: Adaptive Fuzzification

Each of the six concept scores, together with patient age, is converted from a continuous value into three fuzzy membership degrees (LOW, MEDIUM, HIGH) using a triangular membership function. Patient sex, being naturally binary, is not fuzzified and is retained as a single 0/1-valued feature.

The conventional approach in the Fuzzy Pattern Tree literature (Section 2.5) is to fix the triangular membership function bounds uniformly across the observed range of a feature: given a feature with observed minimum and maximum, the LOW set peaks at the minimum, the HIGH set peaks at the maximum, and the MEDIUM set peaks at the midpoint. This scheme implicitly assumes the underlying feature is distributed roughly uniformly or symmetrically around its midpoint; where this assumption is wrong, the resulting low, medium and high terms no longer track genuinely low, medium or high values of the feature, undermining the interpretability that fuzzification is meant to add.

RETINEXPLAIN instead applies the adaptive fuzzification method of de Lima et al. (2025), described in Section 2.5, for every numerical feature. For each feature, three candidate distributions, triangular, exponential and uniform, are fitted to the feature's values on the training set only, to avoid any leakage of test-set distributional information. The distribution with the highest log-likelihood is selected as the best fit for that feature. The peak of the MEDIUM membership function is then set according to the characteristic point of the winning distribution: for a triangular fit, the fitted peak parameter; for an exponential fit, the distribution's mean; for a uniform fit, the midpoint of the range, which recovers the conventional scheme exactly in the case where a feature genuinely is close to uniformly distributed. The membership functions themselves remain triangular in all cases; what is adapted is only the location of the MEDIUM peak, and consequently the slopes of the LOW and HIGH triangles that meet it.

The LOW membership degree equals 1 for values at or below the minimum, falls linearly to 0 at the peak, and remains 0 above the peak. The HIGH membership degree is 0 at or below the peak, rises linearly to 1 at the maximum, and remains 1 above it. The MEDIUM membership degree rises linearly from 0 at the minimum to 1 at the peak, then falls linearly back to 0 at the maximum, following the standard triangular form described in de Lima et al. (2025) but with an adaptively fitted peak rather than a fixed midpoint.

Section 6.3 reports which distribution was selected as the best fit for each of the seven fuzzified features on this project's actual data, and shows that the concept scores are, without exception, better fit by a right-skewed exponential distribution than by a symmetric triangular or uniform one, which is direct empirical justification for using this adaptive scheme rather than the conventional uniform one in this specific application.

After fuzzification, each image is represented by 22 features: six concepts each expanded into three fuzzy sets (18 features), age expanded into three fuzzy sets (3 features), and sex retained as a single binary feature (1 feature).

4.4 Stage 4: Fuzzy Pattern Tree Representation and Genetic Programming Search

The final classifier is a Fuzzy Pattern Tree, represented as a typed Genetic Programming tree following de Lima et al. (2024) and implemented directly in DEAP rather than through a third-party FPT library. The tree's function set consists of the fuzzy operators listed in Table 1: minimum, maximum, weighted average (WA), ordered weighted average (OWA), dilator, concentrator and complement, each operating on values in $[0, 1]$ and returning a value in $[0, 1]$. The weighted average and ordered weighted average operators additionally take a constant weight parameter, restricted to the set $\{0.1, 0.2, \dots, 0.9\}$ following de Lima et al. (2024). The tree's terminal set is the 22 fuzzified features described in Section 4.3.

Because this is a seven-class problem, the tree's root node is a Winner-Takes-All operator with seven branches (WTA7), one per class (Normal, Diabetic Retinopathy, Glaucoma, Cataract, AMD, Hypertension, Myopia), following the multi-class formulation established in de Lima et al. (2024) and used by them, for example, on their seven-class "segment" benchmark dataset. Each of the seven branches is itself a fuzzy expression built from the operators and terminals above, and the WTA7 node simply returns the vector of all seven branch values; the predicted class is the argmax of that vector.

Table 1: Fuzzy operators used in the RETINEXPLAIN function set.

Operator	Definition
MINIMUM(a, b)	$\min(a, b)$
MAXIMUM(a, b)	$\max(a, b)$
WA(a, b, r)	$r \cdot a + (1 - r) \cdot b$
OWA(a, b, r)	$r \cdot \max(a, b) + (1 - r) \cdot \min(a, b)$
DILATOR(a)	square root of a

Operator	Definition
CONCENTRATOR(a)	a squared
COMPLEMENT(a)	1 - a

The evolutionary search is configured following the hyperparameters reported as effective in de Lima et al. (2024): population size 500, tournament selection with tournament size 7, crossover probability 0.8, mutation probability 0.05, ramped half-and-half initialisation with minimum initial depth 2 and maximum initial depth 6, and a maximum tree depth of 17. Each independent run evolves a population for 50 generations, and 30 independent runs are executed, each with a different random seed. For each run, the single fittest individual (by training fitness) is retained via a Hall of Fame of size 1 and is then evaluated on the validation set; the individual with the best validation performance across all 30 runs is selected as the final model and evaluated exactly once on the held-out test set.

4.5 Fitness Function: From Weighted RMSE to Normalised Soft Macro F1

Three candidate fitness functions were implemented and compared for evaluating candidate trees during evolution: a weighted RMSE loss with automatically calculated class weights, a soft (differentiable) macro F1 loss, and a soft Matthews Correlation Coefficient (MCC) loss following Gorodkin's (2004) multi-class generalisation of MCC. All three are computed on the same row-normalised probability representation of the tree's branch outputs, described below, and differ only in the loss formula subsequently applied to that representation.

WTA7 prediction depends only on the relative ordering of the seven branch values, not their absolute scale, which creates a specific failure mode if a loss is computed directly on raw branch outputs: an evolutionary search can trivially reduce such a loss by pushing every branch value toward 1 simultaneously, raising every class's soft true-positive count at once without actually differentiating between classes. The resulting trees collapse to predicting the majority class for almost every input, an outcome that looks superficially like convergence but is in fact degenerate. The fix, applied uniformly across all three fitness functions, is to row-normalise the seven branch outputs into a proper probability distribution before computing any loss. Given raw branch values, negative values are clipped to zero and each row is divided by its sum, so that cases where all branches are close to 1 and cases where all branches are close to 0.5 become numerically identical after normalisation. Rows summing to zero are mapped to the uniform distribution of 1/7 per class, which is the correct no-information state.

The weighted RMSE loss computes the squared error between these normalised branch probabilities and a one-hot encoding of the true class, weights each class's squared error by an automatically calculated inverse-frequency class weight, and takes the square root of the mean weighted squared error. The class weight for class k is 1 divided by the number of training examples of class k , rescaled so that the weights sum to the number of classes; this is a principled, data-derived weighting scheme rather than the hand-set alternative, and follows the same logic as scikit-learn's `class_weight='balanced'` option used for the baselines (Section 4.6).

The soft macro F1 loss is computed on the same normalised probabilities, averaging the per-class soft F1 scores across all seven classes equally. The soft MCC loss is defined analogously following Gorodkin (2004), computed from a soft confusion matrix built from the normalised branch probabilities rather than from hard predictions.

All three row-normalised losses were implemented and evaluated across 30 independent runs each (Section 6.4). The row-normalised soft macro F1 loss was selected as the fitness function for the final RETINEXPLAIN model on the basis that it produced the best test macro F1 of the three and was the only configuration to detect all seven classes; both soft MCC and weighted RMSE failed to detect Hypertension. This choice, and the full three-way comparison, is discussed in Section 6.4.

4.6 Baseline Model Configurations

The logistic regression baseline is run in three configurations: patient age and sex alone as a demographic floor; the six raw concept scores together with age and sex; and the full 22-dimensional fuzzified feature representation. All three configurations use balanced class weights to account for the class imbalance in ODIR-5K.

The decision tree and random forest baselines are each fitted twice, once on the eight raw features and once on the 22 fuzzified features, with fine-tuned hyperparameters using three-fold cross-validation on the training set with macro F1 as the scoring criterion. For the decision tree, the search space covers maximum depth, minimum samples per split, minimum samples per leaf and class weight. For the random forest, the search space covers number of estimators, maximum depth, minimum samples per split and class weight.

The EfficientNet-B0 end-to-end baseline uses the same backbone architecture as RETINEXPLAIN's own Concept Bottleneck Layer but trains it fully and directly on the seven disease labels with no concept bottleneck and no interpretability constraint. Unlike the other

baselines it does not use concept or fuzzy features, its purpose is to establish the macro F1 and accuracy a comparable black-box model can achieve from raw images alone, providing the outer bound of the measurable cost of interpretability.

It is worth noting a structural asymmetry in how the FPT and the scikit-learn baselines select their final model. The decision tree and random forest baselines use GridSearchCV with three-fold cross-validation, selecting the hyperparameter configuration that performs best on average across three training folds, which is a conservative central estimate of generalisation. The FPT, by contrast, selects the single best individual across 30 independent stochastic runs evaluated on the held-out validation set. Selecting the maximum of 30 draws is structurally upward-biased relative to a cross-validated mean, in the sense that a lucky run may win even if it is not representative of what the method typically produces. Both procedures are methodologically correct and neither touches the test set, but this asymmetry should be kept in mind when comparing FPT test scores against baseline test scores.

4.7 Post-Hoc Interpretability Analysis

Three further analyses are applied to the evolved Fuzzy Pattern Trees after the Genetic Programming search completes, each intended to substantiate the interpretability claims made about RETINEXPLAIN with concrete, checkable evidence rather than a single qualitative example.

Tree size. The number of nodes in the best individual from each of the 30 independent runs is recorded, giving a mean and spread across runs rather than a single tree size figure. Genetic Programming is well known to be prone to bloat, the uncontrolled growth of tree size without a corresponding gain in fitness (Section 2.5), and a mean tree size is checked against a simple readability criterion before deciding whether additional parsimony pressure is required. If trees are small enough that their per-class branch expressions (below) remain readable in a few lines each, no additional parsimony pressure is applied, since it would only trade away accuracy for a size reduction that is not actually needed.

Feature usage frequency. Across the 30 independent runs, each run's best individual is scanned for which of the 22 fuzzy terminals it actually uses anywhere in its structure. Counting how many of the 30 runs use a given fuzzy feature at least once gives a measure of that feature's usefulness that is robust to the outcome of any single run, since Genetic Programming is stochastic and a single evolved tree could in principle rely on an unrepresentative combination

of features by chance. This count is then cross-referenced against the per-concept Concept Bottleneck Layer AUC (Section 4.2, Section 6.2) to test whether the evolutionary search preferentially uses the concepts the Concept Bottleneck Layer is best able to detect, which would be evidence that the two stages of the pipeline are working coherently together rather than the classifier stage effectively ignoring a poorly performing concept.

Readable branch expressions. The single best tree overall, the one selected by validation macro F1 across all 30 runs and evaluated once on the test set (Section 4.4), is converted into a human-readable form. The WTA7 root node is stripped, exposing its seven children as seven independent per-class expressions, one per disease category. A substitution map replaces each terminal's internal index (IN0 through IN21) with the fuzzy feature name it corresponds to (for example, IN8 becomes `disc_pallor_HIGH`), and a small recursive pretty-printer renders each operator with its familiar name (minimum, maximum, OWA, and so on) rather than DEAP's internal primitive representation. This produces, for each disease class, an expression such as `OWA(disc_pallor_HIGH, NOT(drusen_LOW), 0.7)`: not the idealised "IF...THEN" form sometimes used to describe FPTs informally, but the honest, direct symbolic form of what Genetic Programming actually evolved.

4.8 Statistical Comparison of Fitness Functions

To decide between the three candidate fitness functions evaluated in this dissertation (row-normalised soft macro F1, row-normalised soft MCC, and weighted RMSE, Section 4.5), their 30-run validation macro F1 distributions are compared pairwise using a formal statistical test rather than a direct comparison of means alone. For each of the three pairs of fitness functions (soft macro F1 versus soft MCC, soft macro F1 versus weighted RMSE, and soft MCC versus weighted RMSE), the same procedure is applied. For each of the 30 random seeds, the difference between the pair's validation macro F1 scores is computed, giving 30 paired differences. A Shapiro-Wilk test is then applied to these 30 differences to check whether they are approximately normally distributed. This is the technically correct normality check for a paired t-test, since the paired t-test operates on the differences rather than on the two samples separately. If the differences are normal, a paired t-test is used to test whether their mean differs significantly from zero. If the differences depart from normality, a Wilcoxon signed-rank test is used instead as the non-parametric equivalent. A significance threshold of alpha equals 0.05 is used throughout for all three pairwise comparisons.

Chapter 5: Development

5.1 Development Timeline and Key Design Decisions

RETINEXPLAIN was developed under the guidance of Dr. Douglas Mota Dias and Dr. Allan de Lima, and this section reflects decisions made during this process. It is presented in roughly chronological order, as several design choices were made iteratively while developing the core pipeline.

Prior to initiating Genetic Programming or working on Fuzzy Pattern Trees, particular emphasis was placed on data preparation: verifying that left and right eyes were correctly treated as separate but associated samples; confirming that the six proposed concepts had sufficient grounding in the literature to be valid clinical ideas; performing patient-level data splits; and validating the concept label matrix. The intent was to establish trustworthy tabular features for downstream use, as the quality of the concept scores extracted from images would be critical to all subsequent analysis.

The FPT classifier is a direct implementation of the approach introduced by de Lima et al. (2024), using the DEAP library for Genetic Programming. DEAP was selected due to familiarity with its inner workings from prior coursework, and de Lima's implementation being well-established and directly benchmarked for the particular fitness function and parsimony needs of this project. The maximum tree depth of 17 is adopted from the original paper's experimental settings, as lower values would unnecessarily constrain the search space.

A further scope decision was made mid-development: the automatic extraction of IF/THEN natural language sentences from evolved FPTs, stated as a planned deliverable in the project proposal, was dropped from scope given the time constraints of the project. The information is present in any RETINEXPLAIN output in the form of a readable symbolic expression of the final tree's operations, demonstrated for several patients in Chapter 6. What was removed was only the additional processing step of automatically converting that structure into natural language rule sentences.

5.2 Dataset Preparation: Implementation

The dataset is loaded from ODIR-5K's Kaggle distribution as described in Section 3.3. The labels field is parsed to extract the underlying disease code, and rows labelled Other or with more than one code are removed, leaving 5,684 images in the cleaned dataset.

Each image is loaded by reading the filename column directly, constructing one dataset entry per image with no duplication. The native per-image labels column is used for single-label assignment, sidestepping the multi-label problem by construction.

Concept labels are extracted per image and per eye: the filename is used to determine which eye the image corresponds to, and keyword matching is applied to that eye's diagnostic keyword column only. The resulting concept labels are then joined onto each split using the filename as the join key, rather than by positional index, to avoid the index misalignment bug described in Section 5.6.3.

5.3 Concept Bottleneck Layer: Implementation and Training

The CBL is implemented in PyTorch as described in Section 4.2, using torchvision's EfficientNet-B0 with ImageNet-pretrained weights as the frozen backbone. The trainable concept layer adds 329,478 parameters on top of the frozen backbone's 4,007,548. Images are loaded through a RetinalDataset class applying the augmentation pipeline described in Section 4.2 for training and resize-and-normalise only for validation and test. Training uses BCEWithLogitsLoss with positive class weights capped at 10, Adam optimiser and early stopping monitored on validation loss. In the final run, the best validation loss was reached at epoch 35 and training stopped at epoch 42. All random seeds including PyTorch, CUDA and NumPy are fixed at the start of the notebook to ensure full reproducibility of the CBL training.

5.4 Adaptive Fuzzification: Implementation

The adaptive fuzzification scheme described in Section 4.3 is implemented using scipy.stats. For each of the seven numerical features, stats.triang, stats.expon and stats.uniform are fitted by maximum likelihood on the training split only, and the distribution with the highest log-likelihood is selected. The resulting bounds and peaks are then applied unchanged to the validation and test splits to prevent any leakage of held-out distributional information.

5.5 Fuzzy Pattern Tree Evolution: Implementation

The GP search is implemented in DEAP as described in Section 4.4, using a PrimitiveSetTyped with 22 typed float terminals and the seven fuzzy operators of Table 1. The WTA7 root node is excluded from crossover and mutation. Thirty independent runs are executed, each seeded independently, with tournament selection, subtree crossover and subtree mutation over 50 generations of 500 individuals. The individual with the best validation macro F1 across all 30

runs is retained as the final model and evaluated exactly once on the test set. In the final run this was Run 21, with a validation macro F1 of 0.4099.

5.6 Testing and Debugging

Four distinct implementation bugs were identified and corrected over the course of this project, each of which materially affected reported results at the time it was present, and each of which is documented here in the interest of an honest account of how this pipeline was actually built and validated, rather than presenting only a clean, final version with no record of what went wrong along the way.

5.6.1 The shuffled data loader misalignment.

The first bug was present in the initial fully working version of the pipeline. The training data loader was correctly set to `shuffle=True`, because randomised mini-batch ordering during training helps prevent overfitting. However, the same data loader was later used to extract concept scores from the trained Concept Bottleneck Layer for downstream use in the Fuzzy Pattern Tree stage. Feature extraction is an inference operation and must preserve the input order of samples to ensure consistency with ground-truth labels. Using the shuffled loader resulted in concept scores being returned in a randomly permuted order, rendering all subsequent features incorrectly paired with their true labels. The fix was to introduce a separate data loader with `shuffle=False` used exclusively for feature extraction, leaving the original shuffled loader untouched for training.

The correctness of the fix was validated using a logistic regression baseline as a fast diagnostic. On the broken pipeline, logistic regression on raw concept scores plus age and sex achieved only 15.92% accuracy and a macro F1 of 0.13, while on the fixed pipeline the same experiment achieved 47.07% accuracy and a macro F1 of 0.44, with all seven classes detected. This confirmed that the bug had been genuinely resolved rather than merely masked.

5.6.2 The patient-level image duplication bug.

A previous implementation of the `RetinalDataset` class was based on an incorrect assumption that the rows of the working dataframe represented patients, with `Left-Fundus` and `Right-Fundus` columns representing different eyes, leading to a loop over dataframe rows that read both images per row. However, the working dataframe had already been exploded to one row per image by an earlier step, with a `filename` column identifying exactly which image each row referred to. By not taking this into account, every image was loaded twice, resulting in

duplicated concept scores and inputs to the Fuzzy Pattern Tree stage. The fix was to replace the loop over eye columns with a direct read of the filename column, ensuring each image was loaded exactly once with no duplication.

5.6.3 The index-alignment bug in concept label assignment.

After switching to per-eye, per-image concept extraction, the concept labels were written back onto the already-split dataframes using `.reindex(df_train.index)` against a differently-indexed source dataframe. Because each split had been given a fresh contiguous index via `.reset_index(drop=True)`, this silently pulled concept labels from whichever rows of the source dataframe occupied the same positional index range, rather than the rows corresponding to the correct images. This is the same underlying category of error as the shuffle bug, features and labels recombined by an implicit assumption about index alignment rather than an explicit key. It was caught from an anomalous diagnostic metric: a per-concept AUC of around 0.5 for every concept simultaneously on the validation set, which indicated the labels themselves had been corrupted. The fix was to construct an explicit filename-keyed mapping from the unsplit dataframe and join each split's concept columns onto it by filename rather than by positional index, which recovered per-concept AUC values above 0.82 for three of the six concepts.

5.6.4 The unnormalised fitness collapse.

When the three candidate fitness functions — soft macro F1, soft MCC and weighted RMSE — were first implemented, each was computed on raw unnormalised branch outputs rather than a properly normalised probability distribution. This allowed the evolutionary search to trivially minimise each loss by pushing every branch value toward 1 simultaneously, producing trees that collapsed to predicting only the two majority classes, with the remaining five classes receiving zero precision, recall and F1. The fix, row-normalising branch outputs into a proper probability distribution before computing any of the three losses, is described in Section 4.5 and resolved this collapse for all three fitness functions, restoring detection of all seven classes and materially improving macro F1 (Section 6.4).

5.7 Baseline Model Implementation

The decision tree and random forest baselines are implemented in scikit-learn with fine-tuned hyperparameters, each fitted twice, once on raw features and once on fuzzified features, and evaluated exactly once on the held-out test set.

One methodological detail is worth stating plainly. The baseline notebook trains its own instance of the Concept Bottleneck Layer of identical architecture and configuration, rather than reusing the exact trained weights from the main pipeline. Because all random seeds (PyTorch, CUDA and NumPy) are fixed identically across every notebook (Section 5.3), this independently trained CBL converges to numerically identical concept scores to those in the main pipeline, reaching the same validation loss at the same epoch (Section 6.2). Both runs use the same architecture, loss, class weighting and data splits, so this does not affect the validity of any comparison in Chapter 6.

5.8 Interpretability Analysis Implementation

The tree size, feature usage frequency and readable branch expression analyses are implemented directly within the same notebook as the 30-run GP search, so that each run's best individual is available immediately without reloading or retraining.

Tree size is read using Python's built-in `len()` on the DEAP `PrimitiveTree` object, which returns the total node count. Feature usage is computed by scanning each run's best tree for terminal indices and accumulating a count across all 30 runs of how many runs used each of the 22 fuzzy features at least once. The branch expressions are generated by a small function that substitutes fuzzy feature names for internal terminal indices and applies a recursive traversal over the WTA7 node's seven children, recovering the seven independent per-class expressions reported in Section 6.8.

5.9 Statistical Comparison Implementation

The 30-run validation macro F1 array from each fitness function's Genetic Programming search is saved to disk (as a NumPy `.npy` file) at the end of its notebook run, and the statistical comparison itself (Section 4.8) is performed in a separate lightweight notebook which loads all three of these arrays and applies `scipy.stats.shapiro` for the normality test and then `scipy.stats.ttest_rel` (for paired t-test) or `scipy.stats.wilcoxon` (for Wilcoxon signed-rank test) to perform the comparison. This separation allows one to recompute the comparison in the future (in case of necessity to re-run one of the 30-run fitness function validations) without having to re-compute the entire Genetic Programming search.

5.10 Technology Stack

Table 2 summarises the technology stack used to implement RETINEXPLAIN.

Table 2: Technology stack.

Component	Technology used
Programming language	Python 3.10+
Deep learning framework	PyTorch, torchvision
Visual backbone	EfficientNet-B0 (ImageNet-pretrained, frozen)
Genetic Programming	DEAP
Concept fuzzification	scipy.stats (distribution fitting)
Baselines	scikit-learn (LogisticRegression, DecisionTreeClassifier, RandomForestClassifier, GridSearchCV), torchvision (fine-tuned EfficientNet-B0)
Statistical testing	scipy.stats (Shapiro-Wilk, paired t-test, Wilcoxon signed-rank)
Data handling	pandas, NumPy
Visualisation	matplotlib, seaborn
Development environment	Kaggle notebooks (GPU-accelerated), Jupyter Notebook (.ipynb)

Chapter 6: Results

6.1 Dataset and Split Summary

After cleaning (Section 3.3, Section 5.2), the dataset contains 5,684 images in total. Table 3 gives the class distribution.

Table 3: Class distribution after cleaning.

Class	Images	Percentage
Normal	2,873	50.5%
Diabetic Retinopathy	1,608	28.3%
Cataract	293	5.2%
Glaucoma	284	5.0%
AMD	266	4.7%
Myopia	232	4.1%
Hypertension	128	2.3%

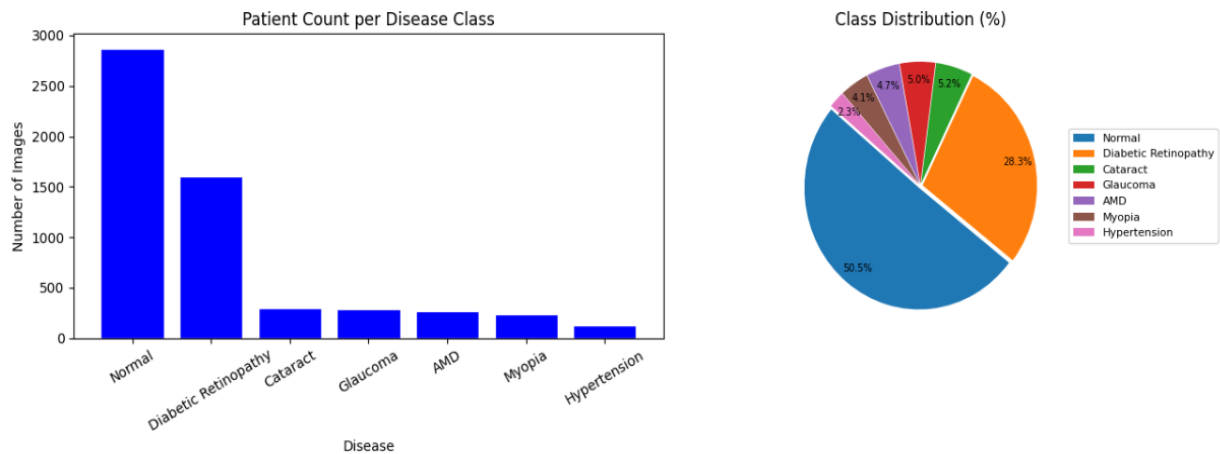


Figure 2: ODIR-5K Class Distribution After Cleaning

The patient level stratified split described in Section 3.6 resulted in a training set of 3,973 images (69.9%), a validation set of 851 images (15.0%) and a test set of 860 images (15.1%). The absence of overlap between patient IDs across all pairs of splits (Train/Validation, Train/Test, Validation/Test) was explicitly checked and verified, satisfying the leak-free requirement described in Section 3.2.

6.2 Concept Coverage, Correlation and Concept Bottleneck Layer Quality

Table 4 reports concept coverage on the training set: the proportion of images whose diagnostic keywords, for that specific eye, matched the keyword list for each concept (Section 3.4, Section 5.2).

Table 4: Concept coverage on the training set (per-image, per-eye).

Concept	Positive samples	Coverage
Haemorrhage	254	6.4%
Drusen	168	4.2%
Disc Pallor	222	5.6%
Vessel Tortuosity	138	3.5%
Red Spot	410	10.3%
Exudate	696	17.5%

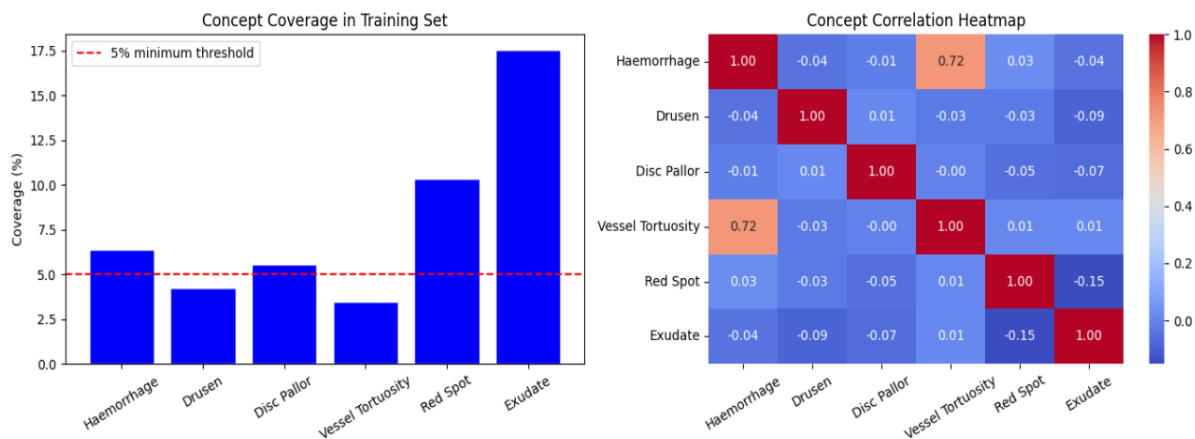


Figure 3: Concept Label Analysis, Training Set

Four of the six concepts exceed the 5% minimum coverage threshold used as a rule of thumb for meaningful supervision; Drusen (4.2%) and Vessel Tortuosity (3.5%) fall just below it, indicating comparatively sparse positive examples for those concepts relative to the others. This is consistent with AMD and Hypertension being among the rarest disease classes overall (Table 3), given that Drusen and Vessel Tortuosity are the concepts most closely associated with those diseases in the keyword extraction rules (Section 3.4).

The Concept Bottleneck Layer was trained as described in Section 5.3, reaching a best validation loss of 0.7912 at epoch 35, with early stopping triggered at epoch 42. Table 5 reports per-concept quality on the validation set after the index-alignment bug described in Section 5.6.3 was corrected. The low precision and recall values reflect the standard 0.5 decision threshold applied against genuinely rare positive classes, not a failure of the underlying score to separate positive from negative cases. The AUC column, being threshold-independent, more directly reflects the CBL's true discriminative ability.

Table 5: Per-concept CBL evaluation on the validation set.

Concept	AUC	Precision	Recall	Positive samples
Haemorrhage	0.867	0.290	0.750	60
Drusen	0.727	0.182	0.286	35
Disc Pallor	0.887	0.240	0.705	44
Vessel Tortuosity	0.848	0.229	0.407	27
Red Spot	0.687	0.145	0.744	86
Exudate	0.678	0.230	0.622	127

Three concepts (Haemorrhage, Disc Pallor, Vessel Tortuosity) exceed an AUC of 0.82, indicating the Concept Bottleneck Layer has learned a genuinely useful visual discriminator for those signs. This is worth reading alongside the validity limitation discussed in Section 2.4 and Section 3.4: Disc Pallor and Vessel Tortuosity are, by construction of the keyword extraction rules, near-deterministic proxies for Glaucoma and Hypertension respectively, so a high AUC for these two concepts in part reflects how learnable those disease labels themselves are from the image, rather than proving the CBL has learned to recognise disc pallor or vessel tortuosity as visually distinct phenomena independent of the disease they imply. Drusen, Red Spot and Exudate, the remaining three concepts, are weaker by this measure (AUC 0.727, 0.687 and 0.678 respectively), which is the more informative signal of the CBL's genuine visual discrimination ability, since Drusen is also a disease-implying keyword-derived concept (Section 3.4) yet achieves a comparatively lower AUC than the other two disease-implying concepts, suggesting AMD-associated drusen may be visually harder to distinguish in this dataset than glaucomatous disc pallor or hypertensive vessel tortuosity.

Table 6 reports the 10th, 50th and 90th percentile of each concept score's distribution on the training set, which is referenced again in Section 6.3 when discussing the shape of these distributions and their consequences for fuzzification.

Table 6: Concept score percentiles (training set).

Concept	P10	P50	P90
Haemorrhage	0.042	0.227	0.664
Drusen	0.041	0.177	0.489
Disc Pallor	0.017	0.146	0.594
Vessel Tortuosity	0.023	0.148	0.497
Red Spot	0.180	0.486	0.676
Exudate	0.164	0.431	0.702

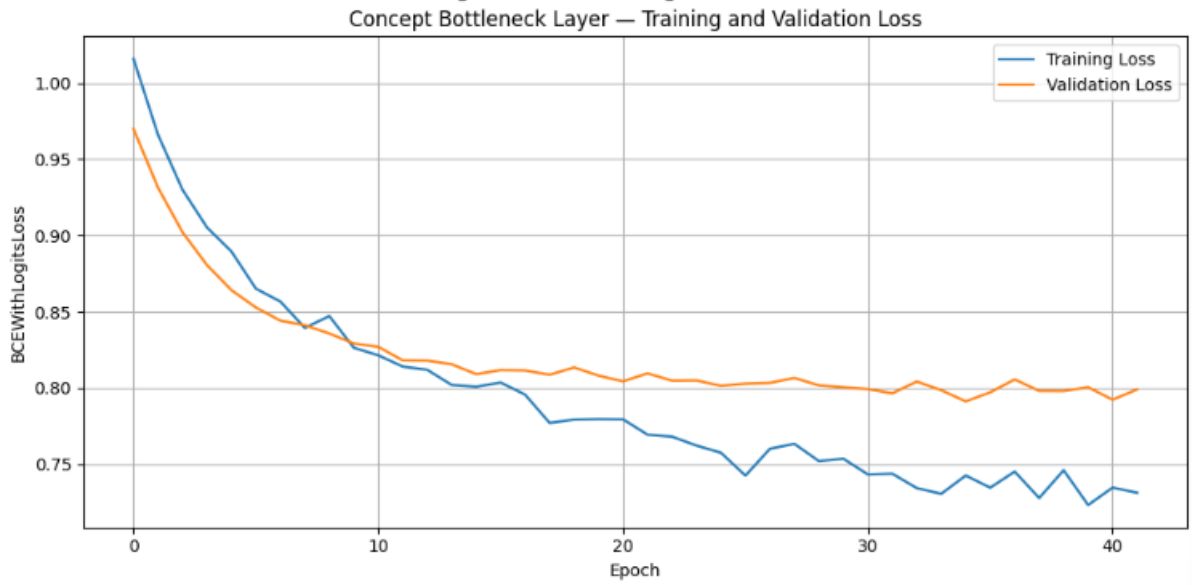


Figure 4: Concept Bottleneck Layer training and validation loss curves.

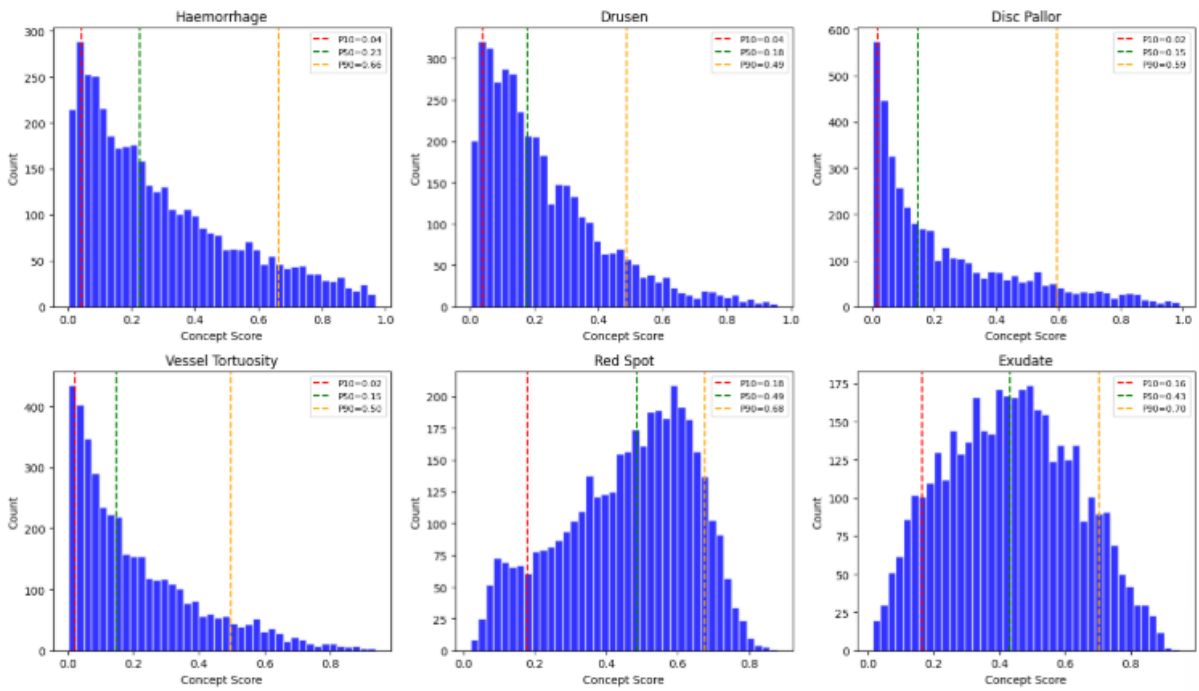


Figure 5: Concept score distributions with P10/P50/P90 percentile markers

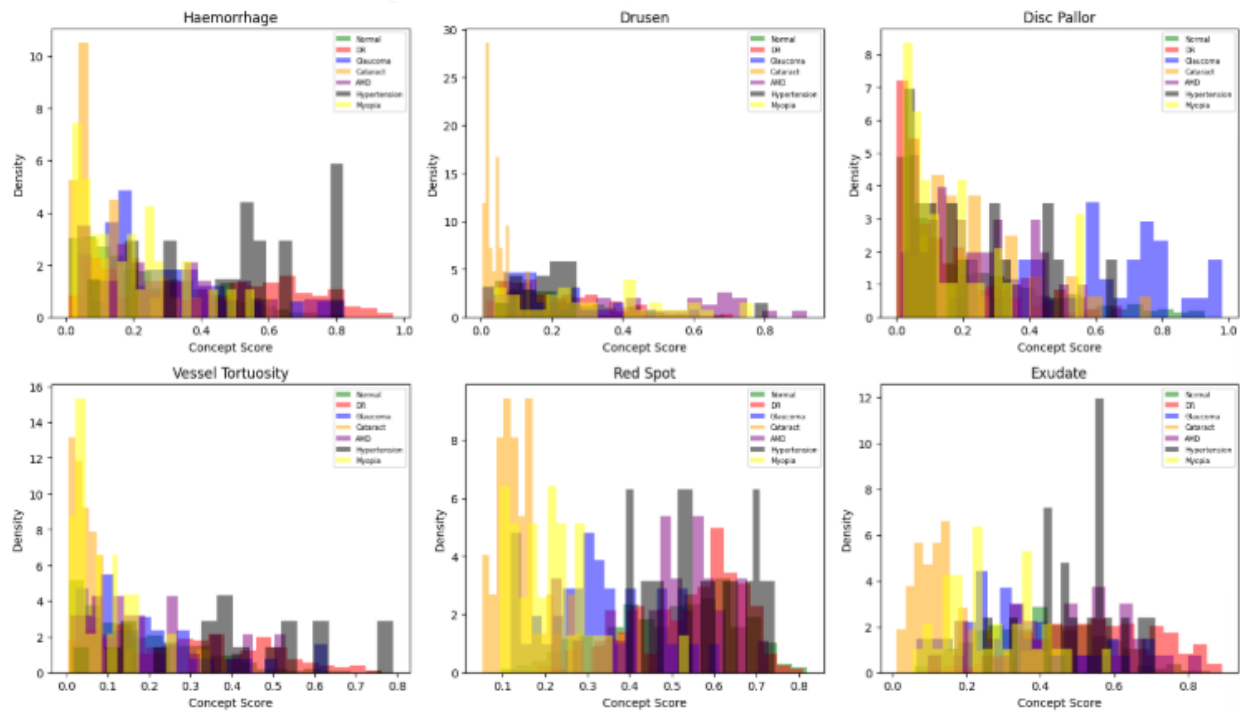


Figure 6: Class-conditional concept score histograms

6.3 Fuzzification Behaviour

Table 7 reports the best-fitting distribution and fitted peak for each of the seven numerical features, selected on the training set as described in Section 4.3 and Section 5.4.

Table 7: Adaptive fuzzification: best-fit distribution and peak per feature.

Feature	Best fit	Fitted peak
Haemorrhage	Exponential	0.293
Drusen	Exponential	0.228
Disc Pallor	Exponential	0.233
Vessel Tortuosity	Exponential	0.208
Red Spot	Uniform	0.450
Exudate	Uniform	0.481
Age	Uniform	46.000

All four of the concept scores derived from the rarer, disease-implying concepts (Haemorrhage, Drusen, Disc Pallor, Vessel Tortuosity) are best fit by an exponential distribution rather than a triangular or uniform one, confirming empirically that these concept scores are strongly right-

skewed, with most images receiving a low score and a comparatively small tail of images receiving a high one; this is directly visible in Table 6, where the P10-to-P50 gap is much narrower than the P50-to-P90 gap for all four of these concepts. This is direct, dataset-specific evidence for the concern raised in de Lima et al. (2025) and discussed in Section 2.5 and Section 4.3: a conventional uniform triangular scheme, which would place the MEDIUM peak at the range midpoint regardless of this skew, would concentrate almost all images into the LOW fuzzy set for these four features and provide little discriminative signal in MEDIUM or HIGH, whereas the adaptive scheme correctly locates the MEDIUM peak near the exponential distribution's mean, closer to where the bulk of the data actually sits. Red Spot, Exudate and Age, by contrast, are best fit by a uniform distribution, for which the adaptive scheme reduces exactly to the conventional midpoint-based scheme, so no expressive power is lost for these three features by using the adaptive method throughout.

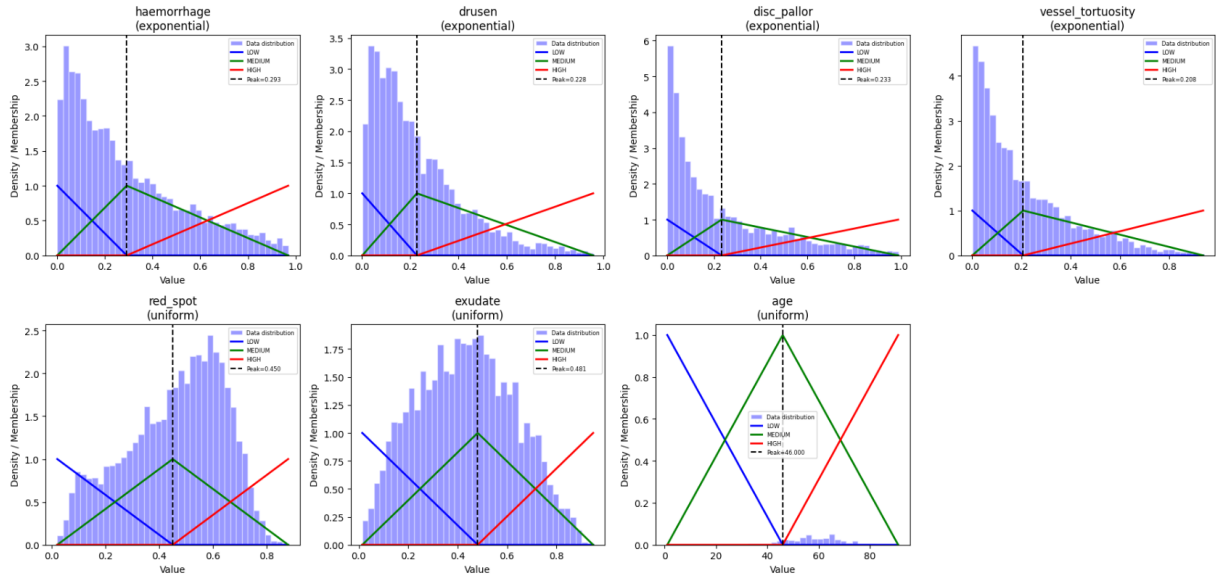


Figure 7: Fuzzy membership function plots overlaid on the empirical data distribution, for each of the seven fuzzified features.

6.4 Genetic Programming Evolution and Fitness Function Comparison

Table 8 compares the three candidate fitness functions evaluated for the Fuzzy Pattern Tree stage, each run for the full 30 independent runs of 50 generations, with the best individual across runs selected by validation macro F1 and evaluated once on the held-out test set.

Table 8: Fuzzy Pattern Tree results by fitness function (test set).

Fitness function	Accuracy	Macro F1	Classes detected (of 7)
Soft macro F1 (row-normalised) (FINAL MODEL)	56.63%	0.4334	7
Soft MCC (row-normalised)	58.02%	0.3803	6 (misses Hypertension)
Weighted RMSE (auto inverse-frequency weights)	57.44%	0.4163	6 (misses Hypertension)

All three row-normalised fitness functions - soft macro F1, soft MCC and weighted RMSE - produce working, differentiated classifiers across all or most of the seven classes, in contrast to the unnormalised versions described in Section 5.6.4 which collapsed to predicting only the two majority classes. The final chosen model is the row-normalised soft macro F1 fitness function, on the basis that it achieves a higher test macro F1 (0.4334 versus 0.4163 and 0.3803) and detects all seven classes, whereas soft MCC and weighted RMSE miss Hypertension entirely. Hypertension is only barely detected under soft macro F1 itself, discussed further in Section 6.5. This choice is further supported by a formal statistical comparison of the fitness functions' 30-run validation distributions, reported in Section 6.10. Soft MCC now achieves the highest raw accuracy of the three (58.02%), but this does not translate into macro F1: the weighted RMSE configuration, despite a lower accuracy (57.44%), reaches a macro F1 of 0.4163, ahead of soft MCC's 0.3803, indicating that its automatically calculated inverse-frequency class weights give it a more balanced signal across classes than soft MCC provides. Weighted RMSE also leaves AMD barely detected (F1 0.089). This confirms that optimising a normalised probabilistic objective does not by itself guarantee minority-class sensitivity; accuracy, macro F1 and per-class recall can each favour a different one of the three configurations, which is itself part of the argument for reporting all three rather than only the winner.

Across the 30 independent runs of the final chosen configuration, the best run was Run 21, which achieved a validation macro F1 of 0.4099. Across all 30 runs, mean validation macro F1 was 0.3415 with a standard deviation of 0.0421.

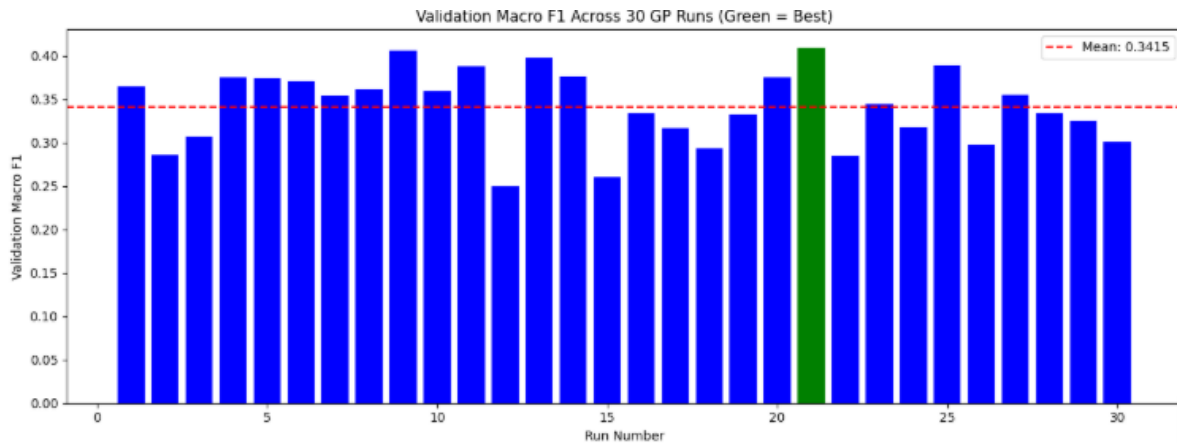


Figure 8: Validation macro F1 across all 30 GP runs, best run highlighted.

6.5 Final Classification Performance

Table 9 compares the final RETINEXPLAIN model (Fuzzy Pattern Tree, row-normalised soft macro F1 fitness) against the other two RETINEXPLAIN Fuzzy Pattern Tree configurations and the full set of baselines described in Section 3.7, all evaluated on the identical 860-image held-out test set.

Before presenting the comparison, it is worth noting the structural asymmetry in model selection described in Section 4.6: the scikit-learn baselines are selected by cross-validated mean, while the FPT is selected as the maximum across 30 stochastic runs on the held-out validation set. Both are valid procedures and neither touches the test set, but the FPT's selection is structurally more optimistic than a cross-validated mean. This asymmetry should be kept in mind when reading the margins in the comparison table below, particularly for the near-tie between RETINEXPLAIN and the decision tree on macro F1.

Table 9: Final model comparison (test set).

Model	Interpretable	Macro F1	Accuracy
Logistic regression, Age + sex only (demographic floor)	Yes (linear)	0.0743	6.63%
Logistic regression, raw concept scores + age + sex	Yes (linear)	0.3872	42.91%
Logistic regression, 22 fuzzy features	Yes (linear)	0.4328	45.93%

Model	Interpretable	Macro F1	Accuracy
Decision tree, raw features	Yes (white-box)	0.4344	49.65%
Decision tree, 22 fuzzy features	Yes (white-box)	0.3747	48.72%
RETINEXPLAIN (Fuzzy Pattern Tree, soft macro F1)	Yes (symbolic/fuzzy)	0.4334	56.63%
RETINEXPLAIN (Fuzzy Pattern Tree, soft MCC)	Yes (symbolic/fuzzy)	0.3803	58.02%
RETINEXPLAIN (Fuzzy Pattern Tree, weighted RMSE)	Yes (symbolic/fuzzy)	0.4163	57.44%
Random forest, raw features	No	0.5106	58.84%
Random forest, 22 fuzzy features	No	0.5003	57.56%
EfficientNet-B0, end-to-end (black box)	No	0.5829	60.12%

The demographic-only floor confirms that age and sex alone carry very little discriminative signal for this seven-class problem (macro F1 of 0.0743, only marginally above what a majority-class predictor would achieve once weighted equally across classes), which in turn confirms that the substantial jump to a macro F1 of 0.3872 once the CBL's concept scores are added is a genuine contribution from the learned visual features, not an artefact of the demographic covariates alone.

RETINEXPLAIN's final model, the Fuzzy Pattern Tree evolved under row-normalised soft macro F1 fitness, achieves a test macro F1 of 0.4334, compared to 0.4344 for the strongest simple interpretable baseline, the raw-feature decision tree. This difference of 0.0010 is negligible and the two models are effectively tied on macro F1. However RETINEXPLAIN outperforms the decision tree on raw accuracy by a seven-point margin: 56.63% against 49.65%, though this accuracy margin, like the macro F1 comparison above, should be read in light of the selection asymmetry noted in Section 4.6, since the FPT's test accuracy also comes from the run selected as best-of-30 on validation. This suggests RETINEXPLAIN classifies the majority classes more correctly while maintaining comparable per-class balance. Both logistic regression variants remain behind on both metrics (0.3872 and 0.4328 macro F1), though the fuzzy-feature variant is now nearly tied with RETINEXPLAIN.

Against the non-interpretable models, the gap is more substantial. RETINEXPLAIN sits 0.0669 macro F1 behind the random forest on the same fuzzy features (0.5003) and 0.1495 macro F1 behind the fully unconstrained EfficientNet-B0 black box (0.5829). These two gaps represent the macro F1 cost of choosing an interpretable model over a non-interpretable one. The random forest, built on the same concept-derived features as RETINEXPLAIN, gives the tighter and fairer estimate. Notably, the random forest achieves 0.5003 macro F1 on the same fuzzy features, which establishes that the concept representation itself is capable of supporting a macro F1 of approximately 0.50; the FPT's lower score of 0.4334 therefore reflects a limitation of the GP search or the FPT representation rather than of the underlying concept features. The end-to-end black box, which discards the concept bottleneck entirely and learns directly from pixels, gives the outer bound. The 0.1495 macro F1 gap to EfficientNet-B0 is substantial and should be read as the measured price of the white-box constraint on this real clinical task, a concrete and quantified cost of choosing full interpretability over unconstrained deep learning on a genuinely difficult seven-class benchmark. Section 2.3 sets out why this cost may be worth accepting in a clinical decision-support context governed by the EU AI Act. The honest reading of these results is that RETINEXPLAIN matches the strongest simple interpretable baseline on macro F1 while exceeding it substantially on accuracy, and this trade-off, rather than a clean win, is what this dissertation reports.

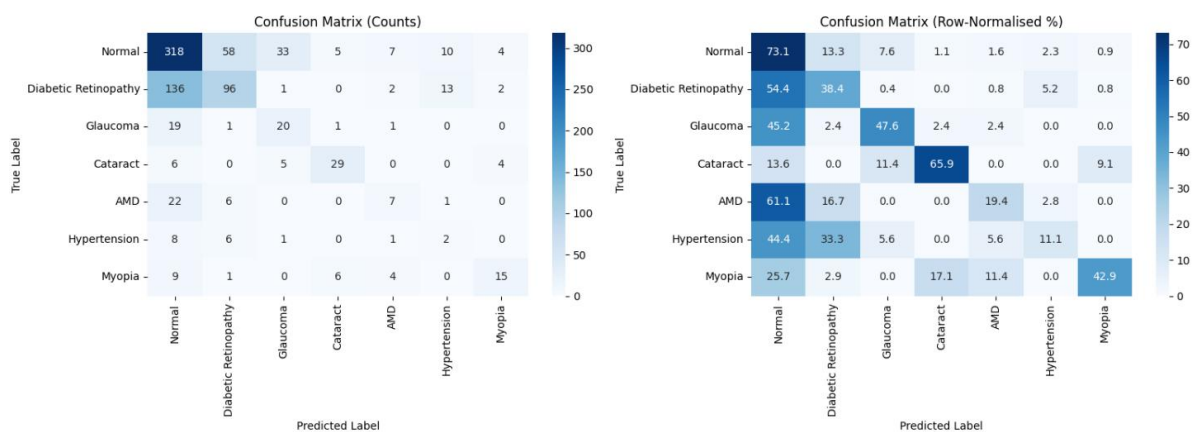


Figure 9: RETINEXPLAIN confusion matrix on the test set (counts and row-normalised percentages).

Table 10 gives the full per-class breakdown for the final RETINEXPLAIN model.

Table 10: RETINEXPLAIN (soft macro F1) per-class performance on the test set.

Class	Precision	Recall	F1	Support
Normal	0.614	0.731	0.667	435
Diabetic Retinopathy	0.571	0.384	0.459	250
Glaucoma	0.333	0.476	0.392	42
Cataract	0.707	0.659	0.682	44
AMD	0.318	0.194	0.241	36
Hypertension	0.077	0.111	0.091	18
Myopia	0.600	0.429	0.500	35

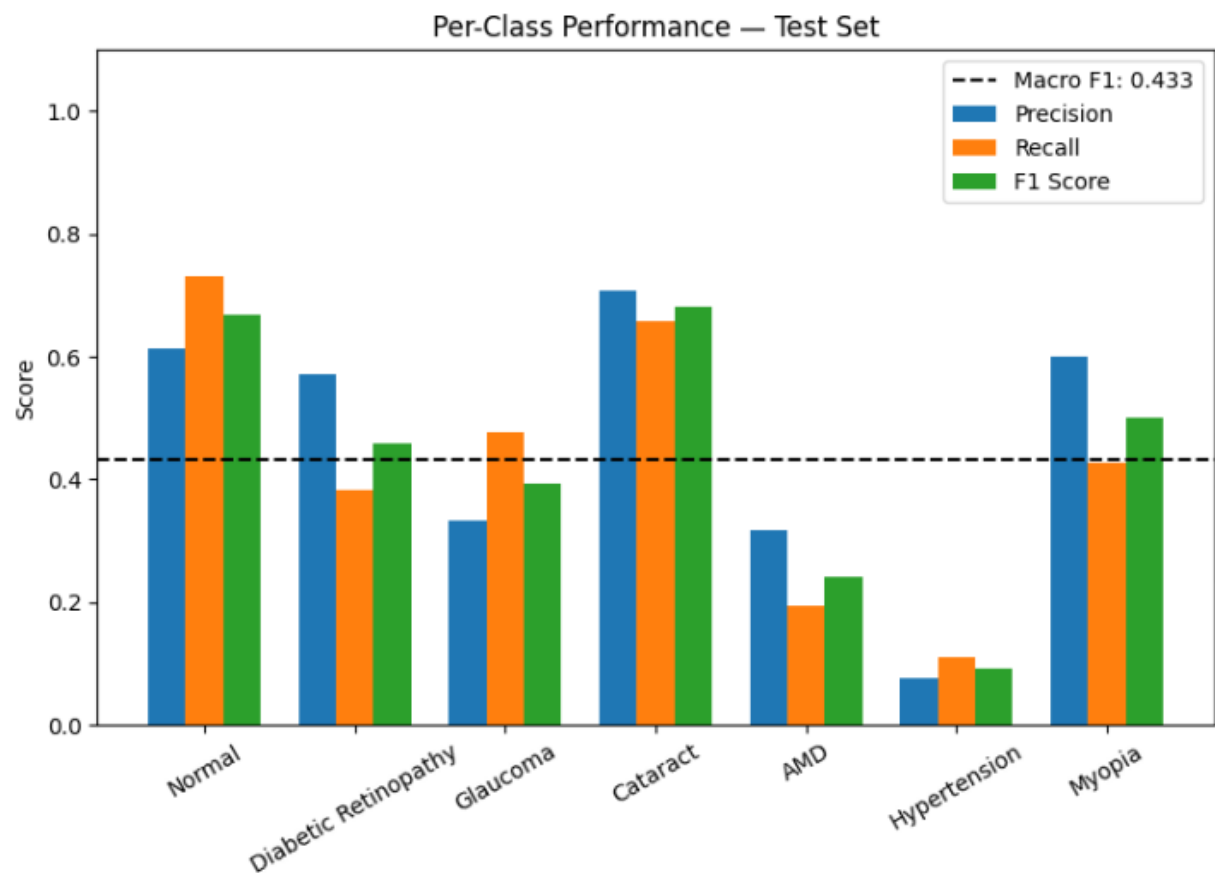


Figure 10: Per-class precision/recall/F1 bar chart with macro F1 reference line

Normal (F1 0.67, recall 0.73) and Cataract (F1 0.68, recall 0.66) are the strongest results, consistent with Normal being the majority class the model has by far the most training signal for, and Cataract being a comparatively distinct visual presentation (clouding of the lens itself, affecting overall image clarity) relative to the other, more subtle retinal signs. Hypertension is the clearest weak point in this particular run: precision (0.077) and recall (0.111) are both low,

meaning the model correctly identified only about 2 of the 18 Hypertension patients in the test set, while also generating enough incorrect Hypertension predictions elsewhere in the test set to pull precision down as well. This dual weakness, rather than a favourable precision/recall trade-off, is itself informative: it suggests the evolved tree's Hypertension branch (Section 6.8) is unreliable rather than conservative. It fires often enough to produce a meaningful number of false positives, yet still fails to isolate most genuine Hypertension cases, indicating the branch condition does not cleanly separate this class from the rest. AMD (F1 0.24) is the other clearly weak class. Both Hypertension and Glaucoma are the smallest classes by support (18 and 42 test samples respectively), and their F1 scores should be read with real caution: with only 18 Hypertension samples in the test set, a difference of a single correctly classified image corresponds to a swing of several percentage points in that class's recall and F1, so a single-point test evaluation on classes this small is close to the limit of what can be measured reliably, a point returned to directly in Section 6.10.

For context, the EfficientNet-B0 baseline (Table 9) achieves a broadly similar pattern across classes (Cataract and Myopia are among its strongest; Hypertension is comparably weak at F1 0.086), which suggests the minority-class difficulty observed for RETINEXPLAIN is at least partly a property of the dataset itself (extreme class imbalance, Section 3.5) rather than being specific to the Fuzzy Pattern Tree architecture. RETINEXPLAIN's Hypertension F1 (0.09) is now nearly identical to the black-box baseline's (0.086) in this run, both reflecting the severe scarcity of Hypertension examples (128 of 5,684 images) rather than a difference in architecture.

6.6 FPT Tree Size

Table 11 reports tree size statistics across the 30 independent runs for all three fitness function configurations, as a prerequisite check for whether additional parsimony pressure (Section 4.7) was required.

Table 11: FPT tree size across 30 runs, by fitness function.

Fitness function	Mean nodes	Std	Min	Max	Best run's tree size
Soft macro F1 (final model)	35.7	13.1	15	79	45 (Run 21)
Soft MCC	52.0	19.6	25	108	50 (Run 24)
Weighted RMSE	44.0	50.3	15	304	42 (Run 27)

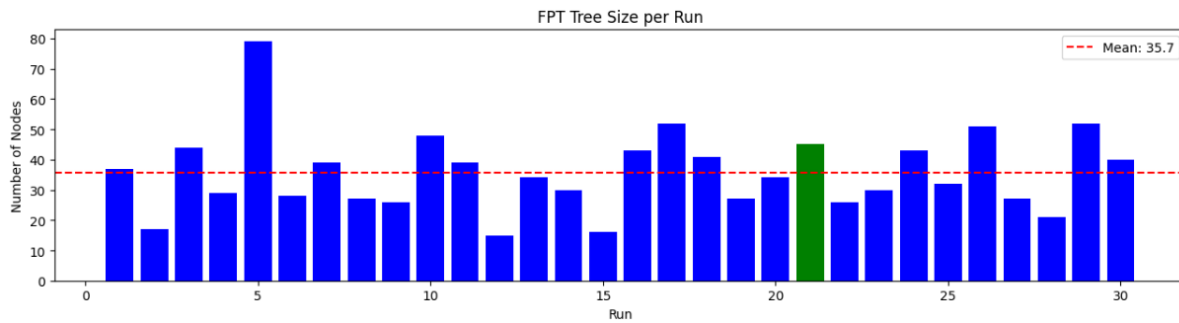


Figure 11: FPT best-individual tree size across 30 GP runs, best run highlighted

The soft macro F1 fitness function produces the smallest mean tree size of the three (35.7 nodes), against 52.0 for soft MCC and 44.0 for weighted RMSE, and it is also the most consistent, with the narrowest standard deviation (13.1) and no run above 79 nodes. Weighted RMSE shows a markedly different pattern: its mean (44.0) sits between the other two, but its standard deviation (50.3) actually exceeds its own mean, driven by a single extreme outlier run that reached 304 nodes, more than four times its mean and nearly three times soft MCC's largest tree. Soft MCC, by contrast, is consistently larger than soft macro F1 across every statistic without any comparably extreme outlier. Judged against the practical readability criterion set out in Section 4.7, the tree size actually selected for the final model was 45 nodes (Run 21), which was assessed as small enough that its per-class branch expressions (Section 6.8) remain readable in a few lines each. Additional parsimony pressure, such as Double Tournament selection, was therefore not applied to the final model.

6.7 Feature Usage Frequency Across 30 Runs

Table 12 reports, for each of the six clinical concepts, how many of the 30 runs' best trees (soft macro F1 configuration) used at least one fuzzy set derived from that concept anywhere in their structure, alongside the concept's own CBL AUC (Table 5) for comparison.

Table 12: Concept-level feature usage across 30 runs versus CBL AUC (soft macro F1 configuration).

Concept	CBL AUC	Total fuzzy-set usages across 30 runs
Disc Pallor	0.887	30/30 (HIGH), 5/30 (LOW), 2/30 (MEDIUM)
Haemorrhage	0.867	30/30 (MEDIUM), 13/30 (HIGH), 5/30 (LOW)
Vessel Tortuosity	0.848	25/30 (HIGH), 3/30 (LOW), 1/30 (MEDIUM)

Concept	CBL AUC	Total fuzzy-set usages across 30 runs
Exudate	0.678	30/30 (HIGH), 9/30 (MEDIUM), 3/30 (LOW)
Red Spot	0.687	30/30 (LOW), 10/30 (MEDIUM), 1/30 (HIGH)
Drusen	0.727	30/30 (HIGH), 12/30 (MEDIUM), 7/30 (LOW)

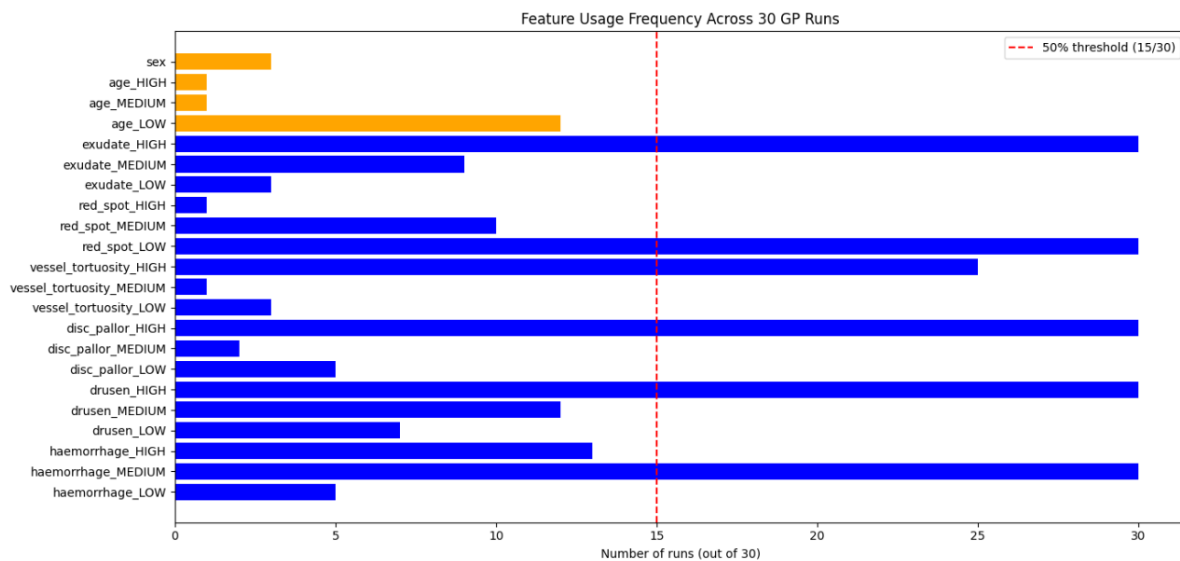


Figure 12: Feature usage frequency across 30 independent GP runs, horizontal bar chart of all 22 fuzzy features.

Five of the six clinical concepts are used by at least one of their three fuzzy sets in all 30 runs (Disc Pallor HIGH, Haemorrhage MEDIUM, Red Spot LOW, Drusen HIGH and Exudate HIGH are each used in every single run), which is a strong signal that Genetic Programming found these concepts useful rather than discarding them. The sixth, Vessel Tortuosity, comes close without quite reaching unanimous use, peaking at 25 of 30 runs for its HIGH fuzzy set. By contrast, age is used in a modest fraction of runs (12 of 30 for LOW, 1 of 30 for MEDIUM, 1 of 30 for HIGH) and sex is used in only 3 of 30 runs, indicating the evolutionary search consistently found the six clinical concepts more useful for classification than the two demographic features. There is a discernible, though not perfect, relationship between a concept's CBL AUC and how consistently the trees use its HIGH fuzzy set: Disc Pallor (AUC 0.887) is used at HIGH in all 30 runs, and Vessel Tortuosity (AUC 0.848) is used at HIGH in 25 of 30. Haemorrhage is a partial exception: despite having the second-highest AUC of any concept (0.867), the trees consistently prefer its MEDIUM fuzzy set (30/30) over its HIGH set (13/30), which is plausible given Haemorrhage's right-skewed distribution (Section 6.3), where

most images score low on this concept and the MEDIUM set may therefore be doing more discriminative work than the comparatively rare HIGH set. Overall, the concepts most consistently used at their clinically meaningful extreme (Disc Pallor HIGH for Glaucoma, Drusen HIGH for AMD, Vessel Tortuosity HIGH for Hypertension) are exactly the three concepts flagged in Section 2.4 as functioning as near-deterministic disease proxies, which is a further independent piece of evidence for that limitation: the evolutionary search has, in effect, discovered and exploited the same proxy relationship identified through direct inspection of the keyword extraction rules.

6.8 Readable Branch Expressions for the Best FPT

The single best tree overall (Run 21, validation macro F1 0.4099, 45 nodes, Section 6.4) is converted into seven per-class expressions using the method described in Section 4.7 and Section 5.8. Table 13 lists each disease class's branch expression, with fuzzy feature names substituted for their internal DEAP terminal indices.

Table 13: Best FPT (Run 21) per-class branch expressions.

Class	Branch expression
Normal	COMPLEMENT(COMPLEMENT(CONCENTRATOR(CONCENTRATOR(red_spot_MEDIUM))))
Diabetic Retinopathy	exudate_HIGH
Glaucoma	CONCENTRATOR(COMPLEMENT(COMPLEMENT(disc_pallor_HIGH)))
Cataract	MAX(CONCENTRATOR(CONCENTRATOR(MAX(CONCENTRATOR(red_spot_LOW), OWA(drusen_HIGH, age_LOW, 0.1)))), MIN(age_LOW, CONCENTRATOR(MAX(CONCENTRATOR(red_spot_LOW), OWA(drusen_HIGH, age_LOW, 0.1)))))
AMD	CONCENTRATOR(drusen_HIGH)
Hypertension	CONCENTRATOR(MIN(DILATOR(MIN(vessel_tortuosity_HIGH, drusen_MEDIUM)), vessel_tortuosity_HIGH))
Myopia	CONCENTRATOR(MIN(red_spot_LOW, CONCENTRATOR(disc_pallor_LOW)))

Three of the seven branches reduce to a single clinical concept each: exudate for Diabetic Retinopathy, disc pallor for Glaucoma, and drusen for AMD. Two of the three additionally wrap the concept in the CONCENTRATOR operator (Glaucoma, AMD), which sharpens an already-high membership degree toward 1, but does not change the qualitative dependence on a single concept; the Glaucoma branch also wraps disc_pallor_HIGH in a double

COMPLEMENT, which cancels out algebraically and is best read as evolutionary bloat rather than a meaningful transformation. Each of these three single-concept rules matches an established clinical association from the ophthalmology literature reviewed in Section 2.4. The evolutionary search, working only from fuzzified numerical scores with no access to disease labels beyond the fitness signal, independently discovered these three well-established disease-to-sign associations without them being encoded anywhere in the function set or search procedure. Read alongside the concept-proxy limitation (Section 2.4, Section 6.7), this finding cuts both ways: it is either evidence that Genetic Programming found genuinely correct clinical rules, or evidence that it exploited the same near-deterministic proxy relationships identified in the keyword extraction rules, or most plausibly some of both.

The Hypertension branch is a partial exception to this pattern: it still centres on vessel tortuosity, the concept flagged in Section 2.4 as its near-deterministic proxy, but combines vessel_tortuosity_HIGH with a secondary drusen_MEDIUM term via MIN and DILATOR rather than depending on vessel tortuosity alone. Since Hypertension has only 18 test-set samples, the smallest of the seven classes, this small addition of an otherwise unrelated concept is at least as plausibly a product of the limited data available to constrain the search as it is a genuine secondary signal.

The Normal, Cataract and Myopia branches are more complex and harder to read directly, built from nested MIN, MAX and OWA operators over a small set of features. The Normal branch says, in effect, that a sharply moderate red spot signal distinguishes Normal from the other classes when no disease-specific concept fires strongly, which only makes sense when read as one branch competing against the other six rather than as a self-contained rule. The Cataract branch is the most feature-rich of the seven, combining red spot, drusen and age signals, while the Myopia branch combines red spot and disc pallor. The Cataract and Myopia branches represent the practical limit of how readable an evolved fuzzy expression can be before formal correctness and human interpretability begin to pull apart.

6.9 Logistic Regression Feature Weights and Three-Way Comparison

Logistic regression coefficients provide a second independent form of interpretability: each fitted weight is directly readable as the strength and direction with which a feature pushes the model's prediction toward or away from a given class. Table 14 reports selected weights from the raw-feature logistic regression model, focusing on each disease class's most clinically relevant concept.

Table 14: Selected logistic regression weights (raw-feature model), by class and its clinically associated concept.

Class	Associated concept	LR weight	CBL AUC	FPT usage (Section 6.7)
Glaucoma	Disc Pallor	6.685	0.887	30/30 (HIGH)
AMD	Drusen	6.867	0.727	30/30 (HIGH)
Hypertension	Vessel Tortuosity	4.224	0.848	25/30 (HIGH)
Myopia	Red Spot	-7.278	0.687	30/30 (LOW)

Table 14 shows a consistent three-way agreement across three entirely independent sources of evidence: the logistic regression weight, the CBL AUC, and the FPT feature usage frequency. Disc Pallor for Glaucoma, Drusen for AMD and Vessel Tortuosity for Hypertension are all strongly positive in the LR weights, well-detected by the CBL, and used at their HIGH fuzzy set in the overwhelming majority of GP runs. The Myopia row shows an equally coherent but different pattern: Red Spot carries the strongest negative weight of any feature (-7.278), meaning a low red spot score is informative for Myopia, and the evolved trees independently converge on exactly this, using `red_spot_LOW` in 30 of 30 runs including directly in the Myopia branch (Table 13). This convergence across three methodologically unrelated analyses is a stronger form of validation for the concept selection than any single analysis would provide alone.

6.10 Statistical Validity

The 30-run validation macro F1 distributions of the three candidate fitness functions (Section 4.5, Section 6.4) were compared using the method described in Section 4.8. Table 15 reports descriptive statistics for all three distributions.

Table 15: 30-run validation macro F1 distributions, by fitness function.

Fitness function	Mean	Std	Min	Max	Median
Soft macro F1	0.3415	0.0421	0.2503	0.4099	0.3493
Soft MCC	0.2989	0.0425	0.1993	0.3876	0.3001
Weighted RMSE	0.2903	0.0622	0.1672	0.4158	0.2998

For each pair of fitness functions, the pairwise difference between validation macro F1 scores was computed across the 30 random seeds, giving 30 paired differences per comparison. A

Shapiro-Wilk test applied to each set of paired differences found all three consistent with normality (Soft F1 vs Soft MCC: $W = 0.9701$, $p = 0.5415$; Soft F1 vs Weighted RMSE: $W = 0.9695$, $p = 0.5262$; Soft MCC vs Weighted RMSE: $W = 0.9807$, $p = 0.8433$), confirming that a paired t-test is appropriate in every case. Soft macro F1 produces significantly higher validation macro F1 than both alternatives: against Soft MCC ($t = 4.0298$, $p = 0.0004$) and against Weighted RMSE ($t = 3.5306$, $p = 0.0014$). The difference between Soft MCC and Weighted RMSE is not statistically significant ($t = 0.6034$, $p = 0.5509$), despite Soft MCC's nominally higher mean (0.2989 versus 0.2903). This gives a quantitative justification for the choice of Soft macro F1 as the final fitness function: its advantage over both alternatives is unlikely to be attributable to run-to-run noise alone, whereas no equivalent claim can be made between Soft MCC and Weighted RMSE.

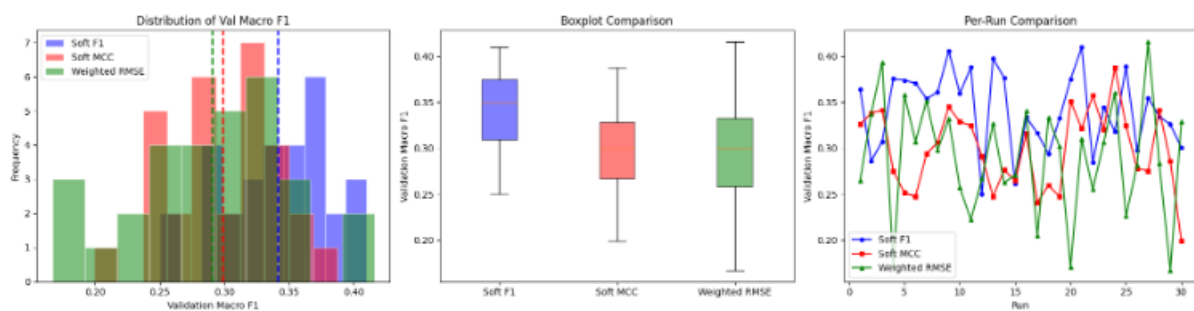


Figure 13: 30-run validation macro F1 distribution comparison (histogram, boxplot, per-run line plot) for Soft F1, Soft MCC and Weighted RMSE

This significance testing addresses the choice of fitness function only. It does not extend to a formal comparison between RETINEXPLAIN and the non-GP baselines, since those are each fitted once rather than across 30 independent stochastic runs and no equivalent paired sample is available. The smallest test classes, particularly Hypertension with only 18 test samples, produce F1 scores that should be read as approximate rather than precise. A repeated-run variance estimate extending the statistical comparison to the full set of baseline comparisons is identified as a direction for future work in Chapter 7.

6.11 Example Clinical Explanations

To illustrate the interpretability RETINEXPLAIN provides, Table 16 shows the concept scores detected for four correctly classified test-set examples, in the form the pipeline produces at inference time.

Table 16: Example correctly classified test-set predictions with detected clinical signs.

Predicted class	Haemorrhage	Drusen	Disc Pallor	Vessel Tortuosity	Red Spot	Exudate
Cataract	LOW (0.02)	LOW (0.02)	LOW (0.11)	LOW (0.01)	LOW (0.09)	LOW (0.05)
Diabetic Retinopathy	HIGH (0.72)	LOW (0.05)	LOW (0.01)	LOW (0.32)	MEDIUM (0.63)	HIGH (0.85)
Glaucoma	LOW (0.24)	LOW (0.08)	HIGH (0.76)	LOW (0.19)	LOW (0.27)	LOW (0.23)
AMD	LOW (0.11)	HIGH (0.74)	LOW (0.04)	LOW (0.07)	LOW (0.21)	LOW (0.28)

The Glaucoma and AMD examples are the cleanest illustrations of the interpretability RETINEXPLAIN aims for: Disc Pallor is detected HIGH (0.76) for Glaucoma while every other concept remains LOW, and Drusen is the only concept detected at HIGH (0.74) for AMD. Both match exactly the single-concept branch rules identified in Section 6.8, and represent clinically coherent explanations a specialist could examine and either endorse or query, something a saliency map from a black-box CNN cannot directly support. The Diabetic Retinopathy example shows Exudate as the strongest detected sign (HIGH 0.85), which is exactly the concept the DR branch depends on (Section 6.8), with Haemorrhage also HIGH (0.72) as a secondary confirmed sign. The Cataract example is notable in a different way: all six concepts score LOW, which is consistent with Cataract being a lens opacity rather than a retinal sign and therefore not expected to activate any of the six retinal biomarkers.

Chapter 7: Conclusion

7.1 Summary of Objectives Met

This dissertation set out to build a working, end-to-end explainable pipeline for multi-class retinal disease classification, combining a deep visual feature extractor, a Concept Bottleneck Layer, an adaptive fuzzy feature representation and a Genetic Programming-evolved Fuzzy Pattern Tree, and to evaluate it honestly against progressively less interpretable baselines. Each of the five objectives set out in Section 1.2 was met. The development process was not without difficulty, four non-trivial implementation bugs were identified and corrected over the course of the project (Section 5.6), each of which materially affected results at the time it was present.

The pipeline itself was successfully built and produces working predictions across all seven target classes for the final chosen configuration (Table 9). Concept labels were derived

automatically from ODIR-5K's existing diagnostic keyword annotations, and their adequacy was assessed empirically through coverage and correlation analysis (Section 6.2) rather than asserted without evidence, though this same analysis surfaced a genuine validity limitation, discussed below, that was not anticipated at the project's outset. The adaptive fuzzification method of de Lima et al. (2025) was applied and shown, on this project's own data, to be materially more appropriate than a conventional uniform scheme, since four of the seven fuzzified features are demonstrably right-skewed rather than uniformly distributed (Table 7). The Fuzzy Pattern Tree classifier was evaluated against a broad set of baselines spanning the interpretability spectrum (Section 6.5), and was found to achieve a macro F1 effectively tied in practical terms with the strongest simple interpretable baseline tested (a tuned decision tree), with a difference of 0.0010 macro F1, well within the run-to-run variance of RETINEXPLAIN's own 30 independent GP runs, while achieving a substantially higher raw accuracy than that baseline, and remaining a modest 0.067 macro F1 behind a tuned random forest built on the same features and 0.150 macro F1 behind a fully unconstrained EfficientNet-B0 black box. All of these gaps, including the near-tie with the decision tree rather than a clean win, are reported exactly as measured rather than rounded in RETINEXPLAIN's favour. The choice among the three candidate fitness functions was additionally supported by a formal statistical test (Section 6.10) rather than an informal preference, and three independent analyses, logistic regression weights, Concept Bottleneck Layer detection accuracy, and evolved-tree feature usage (Section 6.7 and Section 6.9), converged on the same set of clinically established concept-disease associations. The concept-proxy limitation, once identified, was reported directly and its practical consequences examined (Section 6.2, Section 6.7), rather than concealed or minimised.

7.2 Limitations

Three limitations of this work are significant enough to state explicitly, beyond the statistical validity caveats already discussed in Section 6.10.

First, and most fundamentally, three of the six clinical concepts used throughout this project (Disc Pallor, Drusen, Vessel Tortuosity) are, by construction of the keyword extraction rules used to derive their training labels, near-deterministic proxies for Glaucoma, AMD and Hypertension respectively, rather than independently observed visual signs in the sense intended by Koh et al.'s (2020) original Concept Bottleneck Model formulation (Section 2.4). This is a consequence of ODIR-5K providing only free-text diagnostic keywords rather than

independent per-concept visual annotation, not a flaw in the Concept Bottleneck Model architecture itself. This limitation is a documented failure mode of the Concept Bottleneck Model architecture, identified in the broader CBM literature as concept leakage or label entanglement (Mahinpei et al., 2021; Margeloiu et al., 2021); RETINEXPLAIN represents a specific instance of this known failure mode, arising from the keyword-derived concept annotation methodology used rather than from a fundamental flaw in the CBM design. It does mean that a high performance figure for RETINEXPLAIN as a whole cannot be taken as strong evidence that the system has learned to recognise these three signs as visually distinct phenomena; in the worst case, for these three concepts, the system may partly reduce to learning disease classification directly and then re-expressing that classification through a concept score that happens, by the structure of its own training labels, to correlate almost perfectly with the disease it is meant to help explain. This concern is sharpened, not resolved, by the results in Section 6.7 and Section 6.9: the evolved Fuzzy Pattern Trees consistently use exactly these three concepts at their clinically associated extreme, and the logistic regression weights independently agree, which is exactly the pattern that would be observed whether the underlying visual signal is genuine or whether all three analyses are, in effect, rediscovering the same proxy relationship baked into the keyword extraction rules from the outset. This dissertation has been explicit about this limitation throughout, reframing the contribution as "concepts derived from diagnostic keywords" rather than claiming fully independent visual concept learning, but it remains the most consequential limitation of the work as a whole.

Second, the severe class imbalance in ODIR-5K (Section 3.5), with Hypertension comprising only 2.3% of the cleaned dataset and 18 samples in the held-out test set, means the weakest reported result (Hypertension, Table 10) rests on a very small evidence base and should not be treated as a reliable, precise measurement of the system's true performance on that class (Section 6.10); in the final reported run, Hypertension's recall of 0.111 corresponds to correctly identifying only about two patients out of eighteen. AMD (36 test samples, F1 0.24) is the other weak class in this run, though its larger support makes its result somewhat more trustworthy than Hypertension's. This is a property of the dataset rather than the method, but it constrains what can be concluded about RETINEXPLAIN's minority-class behaviour from this single evaluation alone.

Third, fully automated natural-language IF/THEN rule extraction was not implemented (Section 3.2, Section 5.1). The evolved Fuzzy Pattern Tree is interpretable through direct inspection of its symbolic branch expressions, with fuzzy feature names substituted for internal

indices and presented per disease class in Section 6.8. What was not implemented is the additional step of automatically converting those symbolic expressions into plain English IF/THEN sentences. This was a deliberate scope decision made under time constraints, and the branch expressions in Section 6.8 demonstrate that the tree structure is human-readable in its symbolic form, even without a natural-language rendering step.

7.3 Future Work

Several avenues for future work arise directly from the paper's limitations and from unexplored opportunities within the pipeline that were unavailable during this dissertation's timeframe.

Independent concept annotation. The most direct approach to addressing the concept-proxy limitation (Section 7.2) would be to obtain or construct independent visual annotations for the six concepts at the image level, rather than relying on diagnostic keyword text. This would allow a direct test of whether the strong performance on Disc Pallor, Vessel Tortuosity and Drusen genuinely reflects learned visual discrimination of those signs beyond the information encoded in the disease label itself.

Automated natural-language rendering. The symbolic branch expressions produced by RETINEXPLAIN (Section 6.8) present the evolved tree's logic in readable fuzzy operator notation, and the per-patient concept score table (Section 6.11) allows a clinician to see which signs were detected and at what strength. What remains unimplemented is the additional step of automatically converting the branch expressions into plain English IF/THEN sentences, which would make the rule structure accessible without any familiarity with fuzzy operator notation. This would complete the interpretability pipeline as originally envisaged in the project proposal.

Broader parsimony pressure comparison. This dissertation adopted tournament selection with hyperparameters reported by de Lima et al. (2024) as effective across their benchmark datasets, and replaced their RMSE fitness function with a row-normalised soft macro F1 loss. A natural avenue for future work would be to test whether the Lexicase-based parsimony pressure methods described by de Lima et al. (2024, 2025), benchmarked against RMSE, offer a better accuracy-versus-size tradeoff under the soft macro F1 fitness landscape used here.

Minority class strategies. Beyond the class-weighted loss applied in the Concept Bottleneck Layer (Section 4.2), targeted modifications to the Fuzzy Pattern Tree evolution process, such as stratified sampling of fitness cases or a fitness function with explicit per-class recall targets,

could be explored as a way of addressing the performance gap observed for Hypertension and AMD in particular (Table 10).

7.4 Closing Remarks

RETINEXPLAIN demonstrates that a deep visual feature extractor, a clinically grounded Concept Bottleneck Layer, an adaptive fuzzy feature representation and a Genetic Programming-evolved Fuzzy Pattern Tree can be combined into a single working pipeline for a genuinely difficult multi-class medical imaging problem, and that every prediction the resulting system makes can be traced through named clinical concepts to an explicit, human-readable symbolic rule. Among the interpretable models evaluated in this dissertation, it achieves a macro F1 effectively tied in practical terms with the strongest simple alternative, with a difference of 0.0010 macro F1 well within the run-to-run variance of RETINEXPLAIN's own 30 independent GP runs; against the fully unconstrained black-box baseline, its performance falls 0.1495 macro F1 short, a gap that represents the measured price of the white-box interpretability constraint on this real clinical task, and this dissertation has been deliberate about reporting both of these comparisons exactly as measured, including the near-tie rather than the clean win, together with a genuine and consequential limitation in how the underlying clinical concepts were derived. Taken together, this project's contribution is not a claim that any individual component (Concept Bottleneck Models, adaptive fuzzification, Genetic Programming-evolved Fuzzy Pattern Trees) is itself novel; each is drawn directly from existing, cited work. The contribution is in combining them, for what is, to the best of my knowledge, the first time, into a single interpretable pipeline for image-derived retinal disease classification, and in reporting, without overstatement, both what that combination achieves and where it falls short.

References

- Annunziata, R., Garzelli, A., Ballerini, L., Mecocci, A. and Trucco, E. (2016) 'Leveraging multiscale Hessian-based enhancement with a novel exudate inpainting technique for retinal vessel segmentation', *IEEE Journal of Biomedical and Health Informatics*, 20(4), pp. 1129-1138.
- de Lima, A., Carvalho, S., Mota Dias, D., Amaral, J., Sullivan, J.P. and Ryan, C. (2024) 'Fuzzy Pattern Trees for Classification Problems Using Genetic Programming', in Giacobini, M., Xue,

B. and Manzoni, L. (eds.) Genetic Programming: Proceedings of EuroGP 2024. Lecture Notes in Computer Science, vol. 14631. Cham: Springer, pp. 3-20.

de Lima, A., Albarracín, J.F.H., Mota Dias, D., Amaral, J. and Ryan, C. (2025) 'On fitting numerical features into probabilistic distributions to represent data for fuzzy pattern trees', Genetic Programming and Evolvable Machines, 26, article 25.

dos Santos, A.R. and Amaral, J.L.M. (2015) 'Synthesis of Fuzzy Pattern Trees by Cartesian Genetic Programming', Mathware and Soft Computing: The Magazine of the European Society for Fuzzy Logic and Technology, 22(1), pp. 52-56.

European Parliament and Council of the European Union (2024) Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act). Official Journal of the European Union.

Gorodkin, J. (2004) 'Comparing two K-category assignments by a K-category correlation coefficient', Computational Biology and Chemistry, 28(5-6), pp. 367-374.

Huang, Z. and Gedeon, T.D. (2006) 'Pattern Trees', in 2006 IEEE International Conference on Fuzzy Systems. Vancouver: IEEE, pp. 1784-1791.

Huang, Z., Gedeon, T.D. and Nikraves, M. (2008) 'Pattern trees induction: a new machine learning method', IEEE Transactions on Fuzzy Systems, 16(4), pp. 958-970.

Hussein, S.A., Farouk, A.A. and Saeid, M.M. (2025) 'Intelligent retinal disease detection using deep learning', Scientific Reports, 15, article 43282.

Koh, P.W., Nguyen, T., Tang, Y.S., Musmann, S., Pierson, E., Kim, B. and Liang, P. (2020) 'Concept Bottleneck Models', in Proceedings of the 37th International Conference on Machine Learning. PMLR 119, pp. 5338-5348.

Koza, J.R. (1992) Genetic Programming: On the Programming of Computers by Means of Natural Selection. Cambridge, MA: MIT Press.

Lundberg, S.M. and Lee, S.I. (2017) 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, 30, pp. 4765-4774.

Mahinpei, A., Clark, J., Lage, I., Doshi-Velez, F. and Pan, W. (2021) 'Promises and pitfalls of black-box concept learning models', arXiv preprint, arXiv:2106.13314. Available at: <https://arxiv.org/abs/2106.13314>

Margeloiu, A., Ashman, M., Bhatt, U., Chen, Y., Jamnik, M. and Weller, A. (2021) 'Do concept bottleneck models learn as intended?', arXiv preprint, arXiv:2105.04289.

Mostafa, K., Hany, M., Ashraf, A. and Mahmoud, M.A.B. (2023) 'Deep learning-based classification of ocular diseases using convolutional neural networks', in 2023 Intelligent Methods, Systems and Applications (IMSA). IEEE. DOI: 10.1109/IMSA58542.2023.10217787

Muchuchuti, S. and Viriri, S. (2023) 'Retinal Disease Detection Using Deep Learning Techniques: A Comprehensive Review', Journal of Imaging, 9(4), article 84.

Murphy, A., Ali, M., Mota Dias, D., Amaral, J., Naredo, E. and Ryan, C. (2020) 'Grammar-based Fuzzy Pattern Trees for Classification Problems', in Proceedings of the 12th International Joint Conference on Computational Intelligence. Budapest: SCITEPRESS, pp. 71-80.

Murphy, A., Ali, M.S., Mota Dias, D., Amaral, J., Naredo, E. and Ryan, C. (2022) 'Fuzzy pattern tree evolution using grammatical evolution', SN Computer Science, 3(6), article 426.

Niemeijer, M., van Ginneken, B., Russell, S.R., Suttorp-Schulten, M.S.A. and Abramoff, M.D. (2007) 'Automated detection and differentiation of drusen, exudates, and cotton-wool spots in digital color fundus photographs for diabetic retinopathy diagnosis', Investigative Ophthalmology and Visual Science, 48(5), pp. 2260-2267.

ODIR-2019 Grand Challenge (2019) Ocular Disease Intelligent Recognition. Available at: <https://odir2019.grand-challenge.org/> (Accessed: 2026).

Ribeiro, M.T., Singh, S. and Guestrin, C. (2016) '"Why should I trust you?": Explaining the predictions of any classifier', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, 13-17 August 2016, pp. 1135-1144.

Selvaraju, R.R., Cogswell, M., Das, A., Vedantam, R., Parikh, D. and Batra, D. (2017) 'Grad-CAM: Visual explanations from deep networks via gradient-based localization', Proceedings of the IEEE International Conference on Computer Vision (ICCV), Venice, 22-29 October 2017, pp. 618-626.

Senge, R. and Hüllermeier, E. (2011) 'Top-down induction of fuzzy pattern trees', IEEE Transactions on Fuzzy Systems, 19(2), pp. 241-252.

Tan, M. and Le, Q.V. (2019) 'EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks', in Proceedings of the 36th International Conference on Machine Learning. PMLR 97, pp. 6105-6114.

World Health Organization (2019) World Report on Vision. Geneva: World Health Organization.

Appendices

Appendix A: Concept Keyword Extraction Rules

The following keyword lists were used to derive the six clinical concept labels from ODIR-5K's diagnostic keyword text (Section 3.4, Section 5.2), applied per eye using that eye's own diagnostic keyword column.

```
CONCEPT_KEYWORDS = {
    'haemorrhage': [
        'intraretinal hemorrhage', 'severe nonproliferative retinopathy',
        'proliferative diabetic retinopathy', 'hypertensive retinopathy',
        'branch retinal vein occlusion', 'central retinal vein occlusion'
    ],
    'drusen': [
        'drusen', 'dry age-related macular degeneration'
    ],
    'disc_pallor': [
        'optic nerve atrophy', 'glaucoma', 'suspected glaucoma'
    ],
    'vessel_tortuosity': [
        'vessel tortuosity', 'hypertensive retinopathy', 'white vessel'
    ],
    'red_spot': [
        'mild nonproliferative retinopathy', 'diabetic retinopathy',
        'intraretinal microvascular abnormality'
    ],
    'exudate': [
        'maculopathy', 'wet age-related macular degeneration',
        'moderate non proliferative retinopathy'
    ]
}

def extract_concepts(text):
    if pd.isna(text): text = ""
    text = text.lower()
```

```
return {c: 1 if any(kw in text for kw in kws) else 0
        for c, kws in CONCEPT_KEYWORDS.items()}
```

Appendix B: Concept Bottleneck Model Architecture (PyTorch)

```
class ConceptBottleneckModel(nn.Module):
    def __init__(self, num_concepts=6):
        super().__init__()
        backbone = efficientnet_b0(weights=EfficientNet_B0_Weights.IMAGENET1K_V1)
        self.backbone = nn.Sequential(*list(backbone.children())[:-1])
        for param in self.backbone.parameters():
            param.requires_grad = False
        self.concept_layer = nn.Sequential(
            nn.Flatten(),
            nn.Linear(1280, 256),
            nn.ReLU(),
            nn.Dropout(0.5),
            nn.Linear(256, num_concepts)
        )

    def forward(self, x):
        features = self.backbone(x)
        return self.concept_layer(features)
```

Appendix C: Row Normalisation and Soft F1 / Soft MCC / Weighted RMSE Fitness Functions

```
def normalise_probs(branch_values):
    """Row-normalise branch outputs so each sample's branches sum to 1."""
    clipped = np.clip(branch_values, 0.0, None)
    row_sums = clipped.sum(axis=1, keepdims=True)
    return np.where(row_sums > 0, clipped / row_sums, 1.0 / branch_values.shape[1])

def soft_f1_loss(branch_values, Y):
    branch_values = normalise_probs(branch_values)
    tp = np.sum(branch_values * Y, axis=0)
    fp = np.sum(branch_values * (1 - Y), axis=0)
    fn = np.sum((1 - branch_values) * Y, axis=0)
    soft_f1_per_class = 2 * tp / (2 * tp + fp + fn + 1e-8)
    return 1 - np.mean(soft_f1_per_class)

def soft_mcc_loss(branch_values, Y):
    y_proba = normalise_probs(branch_values)
    Y = np.asarray(Y, dtype=float)
    # Soft confusion matrix
    C = Y.T @ y_proba
    N = C.sum()
    t_k = C.sum(axis=1)
    p_k = C.sum(axis=0)

    numerator = N * np.trace(C) - np.dot(p_k, t_k)
    denominator = np.sqrt((N ** 2 - np.dot(p_k, p_k)) * (N ** 2 - np.dot(t_k, t_k)))
    mcc = 0.0 if denominator == 0 else float(numerator / denominator)
    return 1 - mcc
```

```
def weighted_rmse_loss(branch_values, Y):
    Y = np.asarray(Y, dtype=float)
    y_proba = normalise_probs(branch_values)
    # Auto inverse-frequency weights
    class_counts = Y.sum(axis=0)
    weights = 1.0 / (class_counts + 1e-8)
    weights = weights / weights.sum() * len(weights)
    squared_errors = (y_proba - Y) ** 2
    weighted_errors = squared_errors * weights
    rmse = np.sqrt(weighted_errors.mean())
    return float(rmse)
```

Appendix D: Genetic Programming Toolbox Configuration

```
N_RUNS      = 30
GENERATIONS = 50
POPULATION_SIZE = 500
MAX_DEPTH   = 17
MIN_INIT_DEPTH = 2
MAX_INIT_DEPTH = 6
P_CROSSOVER = 0.8
P_MUTATION  = 0.05
TOURNAMENT_SIZE = 7

def WTA7(a, b, c, d, e, f, g):
    return [a, b, c, d, e, f, g]

def create_toolbox():
    pset = gp.PrimitiveSetTyped("MAIN", itertools.repeat(float, n_features), list, "IN")
    pset.addPrimitive(WTA7,      [float] * 7, list)
    pset.addPrimitive(WA,        [float, float, str], float)
    pset.addPrimitive(OWA,       [float, float, str], float)
    pset.addPrimitive(minimum,   [float, float], float)
    pset.addPrimitive(maximum,   [float, float], float)
    pset.addPrimitive(dilator,   [float], float)
    pset.addPrimitive(concentrator, [float], float)
    pset.addPrimitive(complement, [float], float)
    # weight terminals for WA/OWA restricted to {0.1, ..., 0.9}
    # tournament selection (size 7), subtree crossover, subtree mutation
    ...
```

Appendix E: Auxiliary Material Inventory

The following Jupyter notebooks constitute the auxiliary material accompanying this dissertation, to be submitted as the corresponding zip archive.

Notebook	Contents
Retinexplain_softfl_fitness_function.ipynb	Full pipeline: dataset preparation, concept extraction, CBL training, adaptive fuzzification, GP-evolved Fuzzy Pattern Tree (row-normalised soft macro F1), tree size analysis, feature usage analysis, readable

Notebook	Contents
	branch expressions, and example patient explanations. This is the source of the final RETINEXPLAIN results reported in Chapter 6, and the chosen fitness function for the delivered model.
Retinexplain_softmcc_fitness_function.ipynb	Soft MCC fitness function variant, same additional analyses as the Soft F1 notebook, referenced throughout Chapter 6 as a comparison result.
Retinexplain_weightedrmse_fitness_function.ipynb	Weighted RMSE fitness function variant, same additional analyses as the Soft F1 notebook, referenced throughout Chapter 6 as a comparison result.
Baseline_models.ipynb	All baseline models in one notebook: logistic regression (three experiments), decision tree (two experiments), random forest (two experiments), logistic regression weight extraction, and the full baseline comparison summary table.
Statistical_comparison.ipynb	Loads the saved 30-run validation macro F1 arrays from all three main notebooks and performs the Shapiro-Wilk normality test and paired t-tests for all three pairwise comparisons reported in Section 6.10.
Baseline_efficientnet-b0.ipynb	Self-contained end-to-end EfficientNet-B0 black-box baseline.